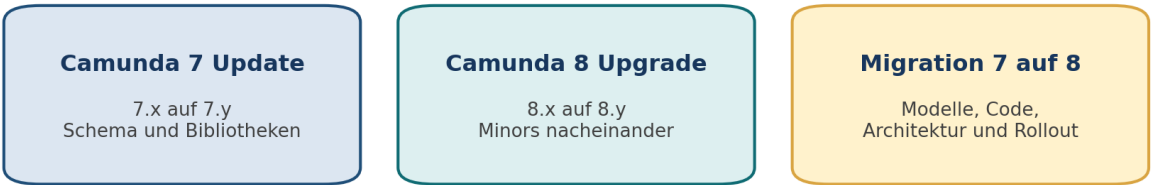


Camunda 7 und Camunda 8

Vergleich, Modernisierung, Upgrade und Migration

Referenz	Zielgruppen	Stand
Camunda 7.24 Camunda 8.9	Architektur Entwicklung Betrieb Fachmodellierung	23. Juni 2026 Offizielle Camunda-Quellen

Drei unterschiedliche Vorhaben



Versionspflege **Versionspflege** **Plattformwechsel**

Ein Plattformwechsel kann nicht durch den Austausch einer Bibliothek erledigt werden.

Abbildung 1: Versionspflege und Plattformmigration sind drei unterschiedliche Vorhaben.

	Zentrale Botschaft Der Wechsel von Camunda 7 zu Camunda 8 ist kein technisches In-place-Upgrade. Er umfasst eine neue Ausführungsarchitektur, andere Integrationsmuster, Modellanpassungen, Code-Refactoring und eine bewusste Rollout-Strategie.
--	---

Schulungsformat: Erklärung, Ursache, Auswirkung, Handlung, Übung

KAPITEL 0

So wird dieses Handbuch genutzt

Das Material ist für Selbststudium, Workshop und Architekturreview aufgebaut. Jedes Kapitel verbindet Begriffe mit Ursachen, Konsequenzen und konkreten Handlungen.

0.1 Lernziele

1. Die Architekturprinzipien beider Plattformen erklären und Unterschiede ursächlich herleiten.
2. Prozessmodelle, Code, Variablen, Formulare und Integrationen auf Migrationsrisiken prüfen.
3. Versionsupdates innerhalb von Camunda 7 und Camunda 8 von der Plattformmigration unterscheiden.
4. Eine passende Strategie für neue und laufende Prozessinstanzen auswählen.
5. Ein Modernisierungsbacklog mit messbaren Gates, Tests und Betriebsnachweisen erstellen.
6. Veraltete oder migrationsfeindliche Muster erkennen und durch tragfähige Alternativen ersetzen.

0.2 Empfohlene Lernpfade

Rolle	Pflichtkapitel	Vertiefung	Ergebnis
Entscheidung und Produktverantwortung	1, 2, 12, 13, 16	11 und 14	Roadmap, Budgetrahmen, Risikobild
Enterprise- und Solution-Architektur	1 bis 16	Anhänge A bis C	Zielarchitektur und Migrationsmuster
Entwicklung	3 bis 10, 14, 15	17	Refactoring- und Testplan
Plattformbetrieb	2, 3, 10 bis 16	Anhänge B und C	Upgrade-, Backup- und Betriebsplan
Fachmodellierung und Analyse	1, 4 bis 7, 9, 13	17	Migrationsfähige Modelle und Abnahmekriterien
Audit, Compliance und Datenschutz	2, 9, 11, 13, 14, 16	Anhang C	Aufbewahrungs- und Nachweisstrategie

0.3 Didaktisches Lesemuster

Begriff Was bedeutet die Funktion oder das Architekturprinzip?	Ursache Welche technische Eigenschaft erzeugt den Unterschied?
Auswirkung Was verändert sich in Modell, Code, Betrieb oder Organisation?	Handlung Welche konkrete Maßnahme reduziert Risiko oder Aufwand?
Nachweis Woran ist erkennbar, dass die Maßnahme abgeschlossen ist?	Übung Wie wird das Wissen auf ein eigenes Beispiel übertragen?

0.4 Kapitelübersicht

Kapitel	Thema	Leitfrage
1	Kernaussagen	Was muss nach zehn Minuten sicher verstanden sein?
2	Lebenszyklus und Support	Welche Zeit- und Supportgrenzen bestimmen die Roadmap?
3	Architektur	Welche Ursachen erklären die meisten Unterschiede?
4	Vergleichsmatrizen	Wie unterscheiden sich Modellierung, Entwicklung und Betrieb?
5 bis 7	Modelle, Ausführung, Daten	Welche Semantik ändert sich im Prozess?
8 bis 10	Code, Aufgaben, APIs und Tests	Was muss konkret refaktoriert werden?
11	Betrieb und Plattform	Welche neuen Betriebsfähigkeiten sind nötig?
12	Updates und Upgrades	Wie werden Versionen kontrolliert aktualisiert?
13 bis 14	Migration und Werkzeuge	Wie erfolgt der Plattformwechsel einschließlich Daten?
15	Anti-Patterns	Was sollte bereits in Camunda 7 nicht mehr neu entstehen?
16	Readiness und Cutover	Welche Gates entscheiden über die Produktionsfreigabe?
17	Übungen und Wissenstest	Ist das Wissen praktisch anwendbar?

Hinweis zur Aktualität

Camunda 8 entwickelt sich schnell. Vor jeder konkreten Umsetzung sind die Release-Ankündigungen, Upgrade-Leitfäden, Supported Environments und Deprecations des Zielreleases erneut zu prüfen.

KAPITEL 1

Kernaussagen in zehn Minuten

Dieses Kapitel bildet das gemeinsame Fundament. Es eignet sich als Management-Zusammenfassung und als Einstieg in eine Schulung.

1.1 Die zehn Regeln

1. Camunda 7 und Camunda 8 sind zwei unterschiedliche Plattformarchitekturen. Das Ziel ist nicht die exakte technische Kopie, sondern eine fachlich gleichwertige und betrieblich tragfähige Lösung.
2. Camunda 7 kann in eine Java-Anwendung eingebettet sein. Camunda 8 wird über remote APIs und Worker angesprochen. Dadurch verschwinden gemeinsame In-process- und Datenbanktransaktionen.
3. Ein Service Task in Camunda 8 wird typischerweise als Job von einem Worker ausgeführt. Jobs können erneut zugestellt werden. Seiteneffekte müssen daher idempotent sein.
4. Camunda 8 nutzt JSON-orientierte Daten und FEEL. Java-Objektserialisierung, Spin-Abhängigkeiten, JUEL-Bean-Aufrufe und implizite Spezialvariablen sind zu vermeiden.
5. Relationale Runtime-Tabellen aus Camunda 7 sind keine Integrationsschnittstelle. In Camunda 8 sind APIs, Ereignisse und unterstützte Exportmechanismen die Systemgrenze.
6. Camunda 7 Updates und Camunda 8 Upgrades erfolgen versionsspezifisch. Die Migration von 7 auf 8 erfordert zusätzlich Modelle, Code, Betrieb und Datenstrategie.
7. Laufende Instanzen werden nicht automatisch durch das Bereitstellen eines Camunda-8-Modells übernommen. Abfluss, Stichtag, Kohorten oder Data Migrator sind bewusst zu wählen.
8. CMMN aus Camunda 7 hat in Camunda 8 keine direkte Fortsetzung. Solche Lösungen benötigen eine Neumodellierung, häufig in BPMN, DMN und Anwendungscode.
9. In Camunda 8 sollten neue Implementierungen die Orchestration Cluster APIs, Camunda User Tasks, Camunda Client und Camunda Process Test verwenden, nicht bereits angekündigte Altpfade.
10. Die beste Vorbereitung beginnt in Camunda 7: saubere Delegates, explizite Schnittstellen, JSON-Daten, kleine Variablen, keine Engine-Interna und keine Geschäftslogik in Ausdrücken.

1.2 Ein Satz pro Perspektive

Perspektive	Camunda 7	Camunda 8	Konsequenz
Architektur	Java-Engine mit relationalem Persistenzkern	Verteilte Orchestrierung mit partitioniertem Ereignisprotokoll	Systemgrenzen neu entwerfen
Integration	Delegate, Connector, REST und eingebettete Aufrufe	Worker, Connectoren, REST und Ereignisse	Remote-Fehler und Wiederholung berücksichtigen
Transaktion	Gemeinsame ACID-Grenze teilweise möglich	Getrennte Commits über Netzwerkgrenzen	Idempotenz und Kompensation
Daten	Beliebige Java-Objekte technisch möglich	JSON-orientierte Werte, Payload-Grenzen	Datenverträge und externe Fachspeicher
Ausdrücke	JUEL, Java Beans und Spin häufig	FEEL auf Daten	Logik aus Ausdrücken entfernen
Betrieb	Relationale DB und Java-Runtime zentral	Cluster, Partitionen, Replikation und Sekundärspeicher	Neue SRE- und Kapazitätskompetenz
Migration	Versionsupdate innerhalb 7.x	Neuaufbau auf Zielarchitektur	Programm statt Bibliothekswechsel

1.3 Schnelltest: Ist die Lösung migrationsfreundlich?

Orientierende Prüfliste

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Sind alle Service-Aufrufe über klar benannte Schnittstellen gekapselt?	Code-Suche, Architekturdiagramm	
2	Sind Variablen JSON-kompatibel und größenbegrenzt?	Variableninventar und Messwerte	
3	Enthalten BPMN-Ausdrücke keine Spring-Bean- oder Java-Methodenaufrufe?	Modellanalyse	
4	Greift kein Anwendungscode direkt auf ACT_RU- oder ACT_HI-Tabellen zu?	Repository- und SQL-Suche	
5	Sind externe Seiteneffekte idempotent oder deduplizierbar?	Integrationstests und Idempotenzschlüssel	
6	Sind Formulare und Task-UIs inventarisiert?	UI- und Formularkatalog	
7	Sind CMMN, Engine-Plug-ins und Cockpit-Plug-ins identifiziert?	Komponentenliste	
8	Ist die Laufzeitverteilung aktiver Instanzen bekannt?	Histogramm nach Alter und Wartezustand	

Quellenbasis: Camunda 7 to Camunda 8 migration guide; Conceptual differences; Migration journey

KAPITEL 2

Lebenszyklus, Support und Zeitdruck

Supportdaten sind keine reine Vertragsinformation. Sie bestimmen Sicherheitsrisiko, verfügbare Patches, Migrationsfenster und die Reihenfolge im Portfolio.

2.1 Referenzstand am 23. Juni 2026

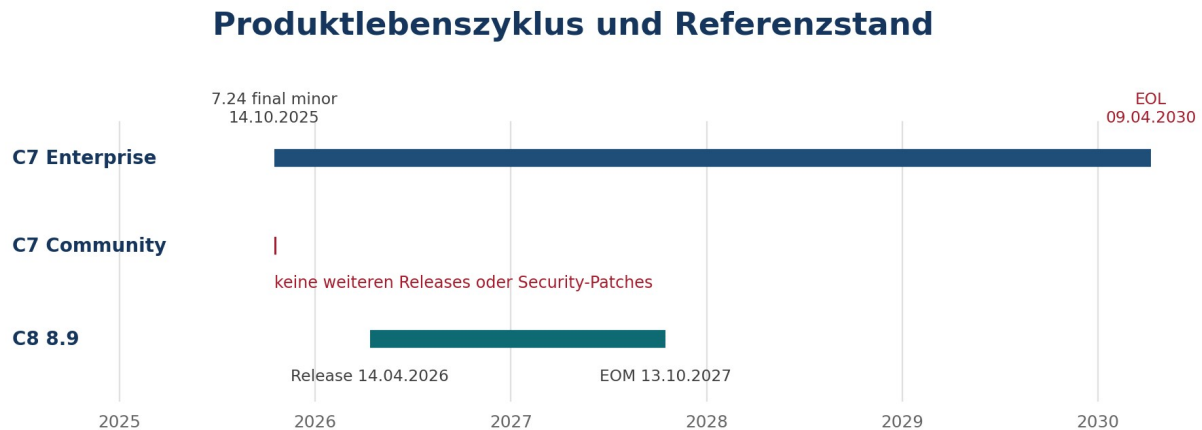


Abbildung 2: Lebenszyklus von Camunda 7 und Referenzrelease Camunda 8.9.

Produktlinie	Relevanter Stand	Supportlage	Planungsfolge
Camunda 7 Enterprise	7.24 ist das letzte Minor Release, veröffentlicht am 14.10.2025	Enterprise End of Life am 09.04.2030; bis dahin begrenzte Patch- und Sicherheitsversorgung gemäß Camunda-Ankündigung	Migration als mehrjähriges Programm planen, nicht bis 2029 verschieben
Camunda 7 Community Edition	7.24 ist das letzte Release	Nach 7.24 keine weiteren Community-Releases und keine Security-Patches	Produktive Risiken und Eigenverantwortung explizit bewerten
Camunda 8.9	Release am 14.04.2026	Geplantes End of Maintenance am 13.10.2027	Upgrade-Fähigkeit als dauerhaften Betriebsprozess etablieren

Ursache des Zeitdrucks

Je länger eine Camunda-7-Lösung unverändert bleibt, desto mehr technische Schuld muss gleichzeitig mit der Plattformmigration abgebaut werden. Die verfügbare Restzeit sinkt, während Abhängigkeiten, Instanzen und Sonderlösungen weiter wachsen.

2.2 Was Support praktisch bedeutet

Sicherheitskorrekturen

Ohne unterstützten Releasepfad muss die Organisation Schwachstellen selbst bewerten, abschirmen oder beheben.

Laufzeitumgebungen

JDK, Datenbank, Application Server, Kubernetes und Suchspeicher müssen in den Supported Environments des

	konkreten Releases liegen.
Fehlerbehebung Supportfähigkeit setzt häufig reproduzierbare Fehler auf einer unterstützten Version voraus.	Upgrade-Kette Ausgelassene Minor Releases erhöhen Analyse- und Testaufwand oder sind bei Camunda 8 nicht zulässig.
Betriebswissen Wissen zu veralteten Komponenten verliert Marktverfügbarkeit. Parallel dazu müssen Camunda-8-Fähigkeiten aufgebaut werden.	Budget Migration konkurriert später mit Supportende, Infrastrukturmodernisierung und Fachänderungen im selben Zeitfenster.

2.3 Deprecation-Radar für neue Camunda-8-Implementierungen

Nicht neu beginnen	Status im Referenzstand	Stattdessen
Operate v1 REST API	Seit 8.8 deprecated, in 8.9 weiterhin verfügbar, Entfernung für 8.10 angekündigt	Orchestration Cluster REST API
Tasklist v1 REST API	Seit 8.8 deprecated, in 8.9 weiterhin verfügbar, Entfernung für 8.10 angekündigt	Orchestration Cluster REST API
Job-based User Tasks für Task-Lifecycle	Abkündigung läuft; Query und Task Management ab 8.10 nicht unterstützt	Camunda User Task
Zeebe Java Client in neuen Anwendungen	Durch Camunda Java Client ersetzt	Camunda Client oder Camunda Spring Boot Starter
Spring Zeebe SDK in neuen Anwendungen	Durch Camunda Spring Boot Starter ersetzt	Camunda Spring Boot Starter
Zeebe Process Test	Seit 8.8 deprecated, Entfernung für 8.10 angekündigt	Camunda Process Test
Direkte Bindung an interne Implementierungsklassen	Keine öffentliche Stabilitätsgarantie	Dokumentierte Public APIs

Planungsregel

Ein Migrationsprojekt sollte nicht nur auf Camunda 8 migrieren, sondern auf einen nicht bereits abgekündigten Integrationspfad des Zielreleases. Sonst entsteht unmittelbar nach dem Cutover das nächste Umbauprojekt.

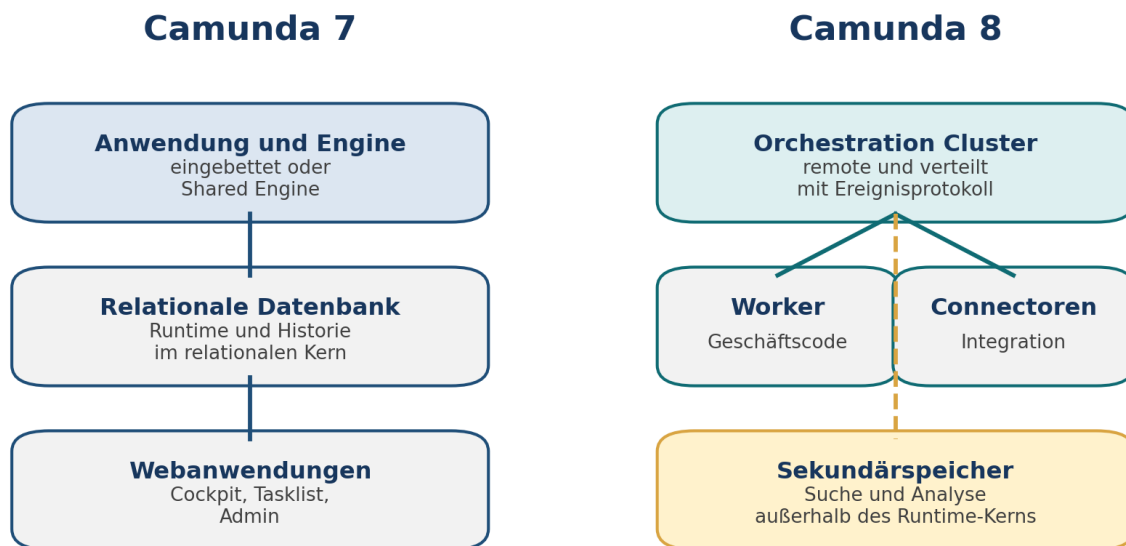
Quellenbasis: Camunda 7 Enterprise EOL Extension Announcement; Camunda 8 release overview; Camunda 8.8 release announcements

KAPITEL 3

Architektur: die Ursache hinter den Unterschieden

Viele Detailunterschiede lassen sich auf drei Ursachen zurückführen: Einbettung gegen Remote-Aufruf, relationale Zustandsänderung gegen Ereignisprotokoll und lokale Transaktion gegen verteilte Kooperation.

3.1 Architekturvergleich



Hauptursache vieler Unterschiede: eingebettete relationale Engine gegen remote verteilte Orchestrierung

Abbildung 3: Vereinfachter Architekturvergleich.

3.2 Die drei Kausalitätsketten

Technische Ursache	Direkte Wirkung	Folgewirkung	Erforderliche Praxis
Engine läuft nicht im Geschäftsprozess der Anwendung	Netzwerk, Latenz und Teilausfälle werden normal	Aufrufe können scheitern oder wiederholt werden	Timeouts, Retries, Idempotenz, Observability
Runtime basiert auf partitioniertem, repliziertem Ereignisprotokoll	Zustand wird als geordnete Folge von Ereignissen aufgebaut	Skalierung und Ausfallsicherheit unterscheiden sich von DB-Cluster-Mustern	Partitionierung, Replikation und Backpressure verstehen
Operative Suche nutzt Sekundärspeicher	Suchansichten können zeitversetzt sein	Command-Erfolg und sofortige Suchsicht sind nicht identisch	Konsistenzbedarf pro API und Use Case definieren
Worker-Commit und Engine-Command sind getrennt	Kein gemeinsamer XA- oder JDBC-Commit	Duplikate oder unvollständige Folgeschritte sind möglich	Outbox, Idempotenzschlüssel, Kompensation
Variablen sind JSON-orientiert	Java-Typen und Klassenpfade sind nicht Teil des Vertrags	Daten werden sprach- und serviceübergreifend nutzbar	Explizite Schemas, Versionierung, Validierung

3.3 Camunda 7 genauer

Camunda 7 ist ein Java-Framework mit Process Engine. Die Engine kann in eine Anwendung eingebettet, als Shared Engine im Application Server oder als Remote Engine über REST betrieben werden. Die Engineknoten

teilen sich typischerweise eine relationale Datenbank. Der Job Executor akquiriert und verarbeitet Jobs gegen diese Datenbank.

Warum direkte Datenbankzugriffe verlockend sind

Die Tabellen sind sichtbar, relational und oft im selben Betriebsbereich. Dadurch wirken SQL-Abfragen wie eine einfache Integrationsmöglichkeit. Tatsächlich sind Schema und Semantik jedoch Engine-Interna und ändern sich mit Versionen. Diese Kopplung verhindert eine saubere Systemgrenze.

3.4 Camunda 8 genauer

Camunda 8 trennt Orchestrierung und Geschäftscode. Das Orchestration Cluster verwaltet Prozesszustand. Worker abonnieren Jobtypen und melden Abschluss oder Fehler zurück. Partitionen bilden persistente, geordnete Ereignisströme und werden zur Ausfallsicherheit repliziert. Operative Such- und Analysefunktionen lesen aus einem Sekundärspeicher.

Begriff	Bedeutung	Betriebliche Relevanz
Partition	Teil des Prozesszustands mit eigenem geordnetem Ereignisstrom	Bestimmt Parallelität, Datenverteilung und Skalierungsgrenzen
Leader und Follower	Replizierte Rollen innerhalb einer Partition	Ermöglichen Failover und beeinflussen Ressourcenbedarf
Gateway oder API-Endpunkt	Zugang für Clients, Commands und Queries	Authentisierung, Routing, Timeouts und Limits
Worker	Externer Ausführer eines Jobtyps	Skaliert unabhängig, braucht Idempotenz und Backpressure
Sekundärspeicher	Such- und Analyseablage für operative Sichten	Kapazität, Retention, Reindexing und mögliche Verzögerung

3.5 Konsistenz: Command ist nicht gleich Query

Ein Command verändert den Runtime-Zustand. Eine Suchabfrage kann auf einer daraus gespeisten, sekundären Sicht arbeiten. Zwischen beiden liegt Verarbeitung. Deshalb muss ein UI oder Integrationstest nicht automatisch davon ausgehen, dass ein unmittelbar zuvor erzeugtes Objekt sofort in jeder Suchabfrage sichtbar ist.

Testregel

Tests auf sekundäre Suchsichten verwenden begrenztes Polling mit fachlichem Timeout. Starre Sleep-Zeiten sind zu langsam oder instabil. Für kritische Steuerung werden APIs mit passender Konsistenzgarantie gewählt.

Quellenbasis: Camunda 7 architecture overview; Camunda 8 conceptual differences; Orchestration Cluster architecture and storage documentation

KAPITEL 4

Vergleichsmatrizen

Die Matrizen dienen als Nachschlagewerk und als Grundlage für Workshops. Eine Zeile beschreibt nicht nur einen Funktionsunterschied, sondern auch die daraus folgende Migrationshandlung.

4.1 Modellierung und Ausführung

Aspekt	Camunda 7	Camunda 8	Migration oder Entscheidung
BPMN-Runtime	Process Engine mit relationalem Zustand	Verteilte, ereignisbasierte Orchestrierung	Semantik und unterstützte Elemente prüfen
Service Task	JavaDelegate, External Task, Connector, Expression	Job Worker oder Connector	Delegate zu Worker refaktorisieren; Typ und Vertrag definieren
External Task	Eigenes Pattern mit Fetch and Lock API	Job Worker ist der native Standard	Fehler-, Retry- und Lock-Semantik neu testen
User Task	Engine-Task mit Tasklist und Forms	Camunda User Task und Tasklist oder eigene UI	Listener, Formulare und Zugriffsmodell prüfen
Script Task	Engine-seitige Script Engines möglich	Keine allgemeine In-engine-Script-Laufzeit wie in C7	Logik in Worker oder Connector verschieben
Business Rule Task	DMN oder externer Entscheidungsaufwurf	DMN über Decision Engine oder Worker	Hit Policies, Variablen und FEEL testen
CMMN	Unterstützt	Keine direkte CMMN-Unterstützung	Neu modellieren oder Fachanwendung einsetzen
Expressions	JUEL, Beans, Spin und Kontextobjekte	FEEL auf Daten	Ausdrücke vereinfachen und Logik entkoppeln
Call Activity	Binding nach Deployment, Version, Tag	Camunda-8-Binding und Propagation	Versionierungsstrategie explizit machen
Incidents	Fehlerzustände in Cockpit	Incidents in Operate und APIs	Runbooks und Ownership anpassen
Timer	DB-basierte Jobs und Job Executor	Verteilter Scheduler im Cluster	Zeit-, Last- und Upgradeverhalten testen
Conditional Events	Unterstützt	Ab Camunda 8.9 verfügbar, Modellsemantik prüfen	Zielrelease festlegen und Konvertierung testen

4.2 Entwicklung und Integration

Aspekt	Camunda 7	Camunda 8	Migration oder Entscheidung
Einbettung	Engine als Bibliothek möglich	Remote-Plattform	Application Boundary neu schneiden
Java API	RuntimeService, TaskService, RepositoryService und weitere	Camunda Client und REST APIs	Service-Aufrufe auf Use Cases abbilden
Engine Plug-ins	Erweiterung interner Engine-Lifecycle möglich	Keine gleichwertige Engine-Plug-in-Schnittstelle	Anforderung über API, Worker oder Eventing lösen
Parse Listener	Modell beim Deployment verändern	Kein direktes Pendant	Modelle vor Deployment transformieren oder Templates verwenden
Execution Listener	Java-Klasse, Delegate Expression, Script	Job-worker-basierte Listener	Nebenwirkungen und Retry-Semantik prüfen

Aspekt	Camunda 7	Camunda 8	Migration oder Entscheidung
Task Listener	Java- oder Expression-basierte Listener	User Task Listener ab neueren C8-Releases	Ereignisse und unterstützte Phasen abgleichen
Connectors	C7 Connect / eigene Konnektoren	Connector Runtime und Element Templates	Connectorwahl, Secrets und Templates neu aufbauen
Messaging	Message Correlation über Engine API	Publish Message mit Correlation Key und TTL	Pufferung, Eindeutigkeit und Duplikate definieren
REST	C7 REST API	Orchestration Cluster REST API plus weitere öffentliche APIs	Endpoint-Mapping und API-Lifecycle berücksichtigen
Events	History Event Handler, Plugins, DB-Auswertung	Exporters, Webhooks, APIs und Events je Produktfunktion	Audit- und Integrationsstrom neu entwerfen
Testing	ProcessEngineRule, camunda-bpm-assert	Camunda Process Test und Integrationstests	Tests neu strukturieren
Frameworks	Java EE, Jakarta EE, CDI, Spring Framework, Spring Boot	Remote Clients und Spring Boot Starter; kein eingebetteter EE-Enginebetrieb	Containerabhängige Integrationen entfernen

4.3 Betrieb, Daten und Governance

Aspekt	Camunda 7	Camunda 8	Migration oder Entscheidung
Runtime-Persistenz	Relationale Datenbank	Partitioniertes, repliziertes Ereignisprotokoll	Backup, Restore und Kapazität neu planen
Operative Suche	DB und Cockpit-Abfragen	Sekundärspeicher und APIs	Eventual Consistency und Retention berücksichtigen
Skalierung	Engineknoten gegen gemeinsame DB	Partitionen, Replikation, Gateway und Worker	Lastmodell und Engpassanalyse ändern
Multi-Tenancy	Tenant IDs in Engine und APIs	Versions- und Deploymentmodell des Zielreleases prüfen	Isolation, Identity und Datenhaltung neu bewerten
Identity	Admin und angebundene Authentisierung	Zentrale Identity- und Admin-Funktionen des Plattformstacks	OIDC, Rollen und Claims planen
Historie	History Level, ACT_HI-Tabellen, Cleanup	Exportierte historische Sichten und Retention	Audit-, Lösch- und Archivkonzept neu erstellen
Backups	Konsistentes DB-Backup	Komponentenspezifischer Backup- und Restore-Plan	Wiederanlauf tatsächlich testen
Monitoring	JMX, Logs, DB, Cockpit	Cluster-, API-, Worker- und Sekundärspeichermetriken	Service Level Indicators definieren
Deployment	Application Server, Spring Boot oder Container	SaaS oder Self-Managed, häufig Kubernetes oder unterstützte Distribution	Betriebsmodell und Verantwortungen festlegen
Lizenz	CE und Enterprise unterscheiden sich	SaaS-Vertrag oder Self-Managed-Lizenz; Produktionsnutzung beachten	Vertrag vor Zielarchitektur klären
Upgrade	DB-Skripte und Bibliotheken, Rolling Update unter Bedingungen	Minor Releases sequenziell; komponenten- und deploymentabhängige Schritte	Regelmäßige Upgrade-Proben etablieren
Datenmigration	Quelle	Ziel für Runtime und optional Historie	Tool-Limits, Downtime und Nachweise prüfen

4.4 Bewertungsskala für das eigene Portfolio

Wert	Bedeutung	Beispiel
0	Kein Befund oder vollständig entkoppelt	Nur primitives JSON und dokumentierte APIs
1	Kleine Anpassung mit automatisierbarem Test	Einfacher JUEL-Vergleich wird zu FEEL
2	Lokales Refactoring eines Modells oder Services	JavaDelegate wird zu Worker
3	Architekturänderung oder mehrere Teams betroffen	Gemeinsame Transaktion wird zu Outbox und Idempotenz
4	Kein direktes Zielkonzept, Neudesign erforderlich	CMMN oder tiefes Engine-Plug-in

Portfolioformel: Für jeden Prozess werden Modell, Code, Daten, UI, Integration, Betrieb und laufende Instanzen getrennt mit 0 bis 4 bewertet. Die Summe priorisiert nicht automatisch, sondern wird mit Geschäftskritikalität, Restlebensdauer und Änderungsdruck kombiniert.

KAPITEL 5

BPMN, DMN und CMMN

Die grafische Ähnlichkeit der Modelle darf nicht mit technischer Gleichheit verwechselt werden. Erweiterungsattribute, Ausführungssemantik und unterstützte Elemente entscheiden über den Aufwand.

5.1 Vier Prüfebenen eines Modells

1. Fachliche Absicht: Welche Zustandsänderung, Wartebedingung und Verantwortung beschreibt das Modell?
2. BPMN- oder DMN-Standardsemantik: Ist die Konstruktion standardkonform oder von einer plattformspezifischen Interpretation abhängig?
3. Camunda-Erweiterungen: Welche camunda:-Attribute, Connectoren, Forms, Listener, Bindings und Expressions sind enthalten?
4. Umgebungsabhängigkeit: Welche Klassen, Beans, Scripts, Secrets, Endpoints oder Datenformate werden zur Laufzeit vorausgesetzt?

5.2 Elementbezogene Migrationsmatrix

Element oder Muster	Typischer Befund in Camunda 7	Ziel in Camunda 8	Prüfung
Service Task mit JavaDelegate	Klassenname oder Delegate Expression	Jobtyp und Worker oder Connector	Ein- und Ausgabe, Retry, Idempotenz, Timeout
External Task	Topic, Lock Duration, Fetch and Lock	Jobtyp, Worker Timeout, Aktivierungsparameter	Wiederholung, Variablenfetch, Fehlerabbildung
Send Task	Delegate, Connector oder Nachricht	Worker, Connector oder Message Publish	Ist fachliche Nachricht oder technische Arbeit gemeint?
Receive Task	Message Subscription	Message Subscription	Correlation Key, Message Name, TTL, Eindeutigkeit
User Task	Form Key, Form Ref, Listener, Candidate Users oder Groups	Camunda User Task, Form, Listener und Assignment	Task-Lifecycle, Zugriff, eigene UI
Script Task	Groovy, JavaScript oder andere Engine	Worker oder Connector; FEEL nur für Ausdrücke und Mappings	Determinismus, Bibliotheken, Sicherheit
Business Rule Task	Decision Ref oder Delegate	DMN oder Worker	Decision ID, Binding, FEEL-Typen
Call Activity	Called Element und Binding	Called Process und Binding	Versionierung, Variablenpropagation, Deploymentreihenfolge
Execution Listener	Klasse, Expression oder Script	Job-worker-basierter Listener	Wiederholung und erlaubte Prozessänderungen
Task Listener	Klasse, Expression oder Script	Camunda User Task Listener, soweit Ereignis unterstützt	Lifecycle-Ereignisse und API-Funktionen
Error Event	BPMN Error oder technischer Fehler	BPMN Error oder Jobfehler	Fachfehler nicht mit Incident verwechseln
Boundary Timer	DB-basierter Job	Cluster-Timer	Zeitzone, Wiederholung, Lastspitzen
Conditional Event	Bedingung bei Variablenänderung	Ab Camunda 8.9 verfügbar	Triggersemantik und Konvertierung
Event Subprocess	Standard plus Camunda-Verhalten	Unterstützung elementbezogen prüfen	Interrupting oder non-interrupting, Variablensicht
Multi-Instance	Sequentiell oder parallel, Collection Expression	Input Collection und Output Collection mit FEEL	Aktive Instanzen, Aggregation, Größenlimits

Element oder Muster	Typischer Befund in Camunda 7	Ziel in Camunda 8	Prüfung
Embedded Subprocess	Lokaler Scope und Listener	Lokaler Scope	Variablenzugriff und Data Migrator Limits

5.3 Modellierungsregeln für migrationsfähige Camunda-7-Modelle

Regel	Ursache	Konkrete Umsetzung
Fachliche und technische Schritte trennen	Ein technischer Delegate enthält sonst mehrere nicht sichtbare Zustandsänderungen	Je externer Seiteneffekt ein klar benannter Schritt oder ein dokumentierter atomarer Worker
Keine Java-Methoden in Expressions	FEEL hat keinen Zugriff auf Spring Beans oder Klassen	Vergleiche und Mappings als Datenexpression; Logik in Service
BPMN Errors nur für fachliche Alternativen	Technische Fehler benötigen Retry und Incident statt fachlichem Routing	Fehlertaxonomie mit Code, Retrybarkeit und Owner
Korrelation explizit modellieren	Die Message-Engine kann fehlende oder mehrdeutige Schlüssel nicht fachlich erraten	Message Name und stabiler Correlation Key als Vertrag
Große Dokumente nicht als Prozessvariable	Payload und Export belasten Runtime und Sekundärspeicher	Dokument extern speichern; Referenz plus Metadaten im Prozess
Call Activities bewusst versionieren	Unklare Bindings erzeugen nicht reproduzierbares Verhalten	Binding-Strategie je Umgebung dokumentieren
Business Key nicht als universellen Primärschlüssel behandeln	Semantik und API-Namen unterscheiden sich; Mehrfachinstanzen können legitim sein	Fachliche IDs im Datenvertrag und eindeutige Korrelation separat definieren

5.4 DMN und FEEL

DMN bleibt eine zentrale Möglichkeit, Entscheidungen vom Ablauf zu trennen. Die Migration muss jedoch nicht nur XML-Kompatibilität, sondern Eingabetypen, Hit Policy, fehlende Werte, Datum und Zeit sowie FEEL-Ausdrücke testen. Ein fachlicher Golden-Master-Test mit repräsentativen Eingaben ist belastbarer als ein reiner Deploymenttest.

Prüfpunkt	Risiko	Testnachweis
Input Expression	C7-Ausdruck greift auf Bean oder Spin zu	Nur Datenzugriff oder vorgelagerte Aufbereitung
Null und fehlende Felder	Andere Behandlung führt zu abweichender Regelwahl	Grenzfallmatrix
Zahlen	Integer, Long und Decimal werden unterschiedlich serialisiert	Typ- und Rundungstests
Datum und Zeit	Zeitzonen oder Stringformate ändern Ergebnis	Fixierte Zeitzone und Formatvertrag
Collect und Aggregation	Reihenfolge oder Typ der Resultate weicht ab	Erwartete Ergebnisliste und Aggregat
Versionierung	Neueste Decision wird unbeabsichtigt genutzt	Deployment- und Bindingtest

5.5 CMMN: Warum Neumodellierung erforderlich ist

Camunda 8 bietet keine direkte CMMN-Runtime als Ziel für Camunda-7-Cases. Die Ursache ist nicht eine fehlende XML-Konvertierung, sondern das fehlende Ausführungsmodell. Milestones, Discretionary Items, Sentries und Plan Items müssen fachlich interpretiert und in andere Bausteine überführt werden.

CMMN-Konzept	Mögliche Zielbausteine	Entscheidungsfrage
Case Plan Model	BPMN-Orchestrierung plus Fachzustand	Ist der Ablauf tatsächlich ad hoc oder nur variantenreich?
Sentry	Conditional Event, Event Subprocess, DMN oder Anwendungsregel	Welche Änderung aktiviert oder beendet einen Schritt?
Discretionary Item	Dynamische User Task, Subprozess oder Fach-UI	Wer darf den Schritt wann hinzufügen?
Milestone	Expliziter Fachstatus, Event oder BPMN-Endzustand	Wird der Zustand nur angezeigt oder steuert er Folgeaktionen?
Case File Item	Externes Domänenobjekt mit Referenzvariablen	Welches System ist fachlicher Dateneigner?

Workshop-Methode

CMMN nicht Element für Element nachzeichnen. Zuerst Ereignisse, Entscheidungsrechte, Fachzustände, Fristen und optionale Aktivitäten extrahieren. Danach Zielmuster auswählen und mit realen Fällen simulieren.

Quellenbasis: Camunda 7 to Camunda 8 migration guide; Diagram Converter; Camunda 8 BPMN and DMN documentation

KAPITEL 6

Ausführungssemantik: Transaktionen, Jobs, Nachrichten und Fehler

Der wichtigste technische Lernschritt ist der Wechsel von einer teilweise gemeinsamen Transaktion zu einer verteilten, wiederholbaren Zusammenarbeit.

6.1 Transaktionsgrenzen

Transaktionsgrenzen und Fehlerbilder

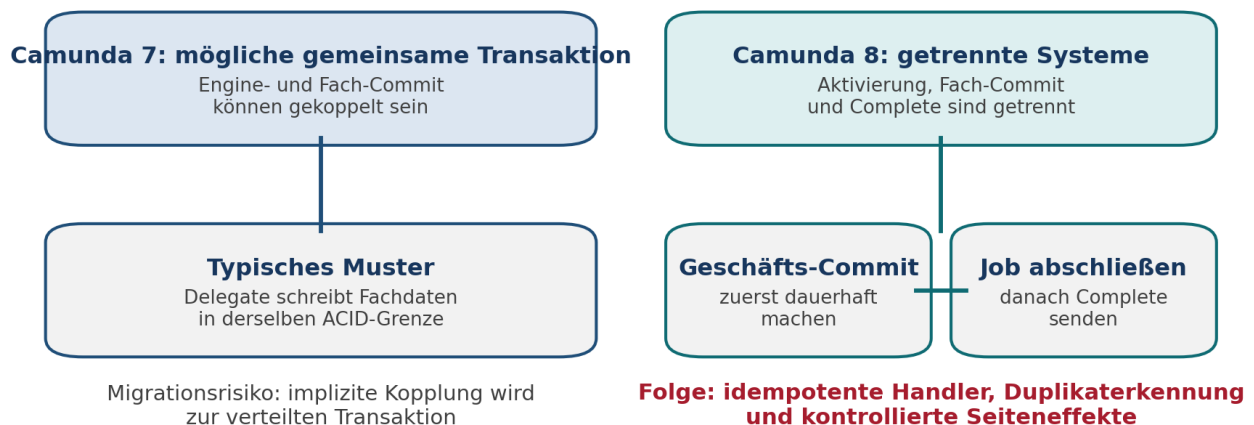


Abbildung 4: Transaktionsgrenzen in Camunda 7 und Camunda 8.

In Camunda 7 kann eine eingebettete Engine denselben Transaktionsmanager und dieselbe Datenquelle wie die Fachanwendung verwenden. Ein Fehler rollt dann Engine- und Fachänderung gemeinsam zurück. In Camunda 8 verarbeitet ein Worker Fachlogik außerhalb des Clusters. Der Fach-Commit und das Complete-Kommando können nicht atomar zusammenfallen.

6.2 Job-Lebenszyklus in Camunda 8

1. Die Prozessinstanz erreicht einen job-erzeugenden Schritt.
2. Das Cluster legt einen Job mit Typ, Variablen, Retries und Timeout an.
3. Ein Worker aktiviert den Job. Die Aktivierung ist zeitlich begrenzt.
4. Der Worker führt Geschäftscode aus und persistiert externe Seiteneffekte.
5. Der Worker meldet Complete, Fail oder einen BPMN Error.
6. Bei Timeout, Verbindungsverlust oder Fehler kann der Job erneut aktiviert werden.
7. Sind keine Retries mehr vorhanden, entsteht typischerweise ein Incident, der operativ behandelt wird.

At-least-once als Ursache

Die Plattform kann nicht sicher unterscheiden, ob ein Worker nach erfolgreichem Fach-Commit vor dem Complete-Kommando ausgefallen ist. Eine erneute Zustellung verhindert verlorene Arbeit, kann aber den Seiteneffekt wiederholen. Deshalb ist Idempotenz keine Optimierung, sondern Teil der Korrektheit.

6.3 Idempotenzmuster

Muster	Funktionsweise	Geeignet für	Grenze
Eindeutiger Idempotenzschlüssel	Empfänger speichert Schlüssel und gibt bei Wiederholung dasselbe Ergebnis zurück	Zahlungen, Bestellungen, Dokumenterzeugung	Empfängersystem muss unterstützen
Upsert auf fachliche ID	Schreiben ist durch eindeutigen Schlüssel deterministisch	Stammdaten und Statusprojektionen	Nicht für additive Buchungen ohne Zusatzlogik
Outbox	Fachänderung und Outbox-Eintrag werden lokal gemeinsam committed; Versand erfolgt separat	Eigene Datenbank plus Messaging	Zusätzlicher Dispatcher und Cleanup
Inbox oder Deduplizierung	Konsument protokolliert verarbeitete Nachricht oder Job-ID	Nicht idempotente Empfänger	Retention und Race Conditions planen
Read-before-write mit Optimistic Lock	Vor Änderung wird Fachzustand geprüft	Zustandsmaschinen	Allein nicht ausreichend bei konkurrierenden Aufrufen
Kompensation	Erfolgreicher Seiteneffekt wird durch fachliche Gegenaktion ausgeglichen	Langlaufende Sagas	Nicht jede Wirkung ist reversibel

6.4 Fehlerklassifikation

Fehlerklasse	Beispiel	BPMN- oder Jobreaktion	Owner
Temporär technisch	HTTP 503, Datenbank kurz nicht verfügbar	Fail mit Retry und sinnvoller Backoff-Strategie	Betrieb oder integrierter Service
Permanent technisch	Ungültige Konfiguration oder fehlendes Secret	Retries begrenzen, Incident und Alarm	Plattform oder Anwendung
Fachlich erwartet	Kredit abgelehnt, Dokument unvollständig	BPMN Error oder modellierter Pfad	Fachprozess
Fachlich unerwartet	Unbekannter Status oder verletzte Invariante	Incident oder kontrollierter Eskalationspfad	Produktteam
Datenvertrag verletzt	Pflichtfeld fehlt, Typ falsch	Früh validieren; je Ursache BPMN Error oder Incident	Sender und Empfänger gemeinsam
Nicht eindeutig	Message korreliert zu mehreren fachlichen Kandidaten	Keine stille Auswahl; Fehler sichtbar machen	Prozess- und Integrationsarchitektur

6.5 Nachrichtenkorrelation

Eine Message in Camunda 8 besitzt einen Namen, einen Correlation Key, optionale Variablen und eine Time to Live. Ist die korrespondierende Subscription beim Publish noch nicht aktiv, kann die Nachricht bis zum Ablauf der TTL gepuffert werden. Eine TTL von null bedeutet unmittelbare Korrelation ohne Pufferung.

Entscheidung	Leitfrage	Fehler bei fehlender Festlegung
Message Name	Welches fachliche Ereignis ist eingetreten?	Technische Endpunkte werden zum Prozessvertrag
Correlation Key	Welche konkrete Instanz oder Entität wartet?	Mehrdeutigkeit oder Nichtkorrelation
Message ID	Wie werden Duplikate innerhalb eines Zeitfensters erkannt?	Doppelte Fortsetzung
TTL	Kann die Nachricht vor der Subscription eintreffen?	Nachricht geht verloren oder bleibt unnötig lange gepuffert
Variablenmapping	Welche Daten gehören zum Ereignis, welche zum Fachsystem?	Große Payloads und unkontrolliertes Überschreiben
Fehlerkanal	Wer erkennt und behandelt nicht korrelierbare Nachrichten?	Unsichtbare Prozessabbrüche

6.6 Timer, Zeit und Last

1. Zeitangaben mit Zeitzone und fachlicher Bedeutung dokumentieren. Ein Kalendertag ist nicht dasselbe wie eine Dauer von 24 Stunden.
2. Wiederholungstimer auf Lastspitzen prüfen. Viele gleichzeitig fällige Jobs können Worker und Downstream-Systeme überlasten.
3. Nachholverhalten nach Downtime testen. Überfällige Timer können konzentriert ausgelöst werden.
4. Business Calendar Logik nicht implizit in Engine-Plug-ins verstecken. Einen expliziten Dienst oder ein nachvollziehbares Modell verwenden.
5. Langfristige Timer in Upgrade-, Backup- und Datenmigrationsproben einbeziehen.

Quellenbasis: Camunda 8 job worker concepts; message correlation; incidents and error handling documentation

KAPITEL 7

Variablen, Datenverträge und Ausdrücke

Prozessvariablen sind Steuerdaten der Orchestrierung, kein Ersatz für ein fachliches System of Record. Diese Trennung reduziert Laufzeitlast und Migrationsrisiko.

7.1 Vergleich der Datenmodelle

Thema	Camunda 7	Camunda 8	Handlung
Grundtypen	Primitive Werte plus serialisierte Objekte und Spin	JSON-orientierte Werte und FEEL-Typen	Auf interoperable Daten reduzieren
Java-Objekte	Kann Klassenname und Serialisierung tragen	Kein Java-Klassenvertrag in Variablen	DTO in JSON-Schema überführen
Dokumente	ByteArray oder serialisierte Werte möglich	Payload-Grenze und Exportkosten	Extern speichern, URI und Metadaten halten
Scope	Execution- und Task-lokale Variablen	Prozess- und lokale Scopes mit Mapping	Variablenlebenszyklus modellieren
Expressions	JUEL und Java-nahe Funktionen	FEEL	Logik vereinfachen und Typen testen
Größenlimit	DB- und Konfigurationsabhängigkeit	Prozessinstanz-Payload einschließlich interner Daten begrenzt; dokumentiert etwa 4 MB	Budget deutlich unter Maximum setzen
Historie	History Level und DB-Tabellen	Sekundärspeicher und Retention	Datenschutz und Cleanup neu planen

7.2 Variablenbudget

Praxisregel

Das technische Maximum ist kein Zielwert. Ein internes Budget pro Instanz, pro Variable und pro Workerantwort schützt Cluster, Netzwerk, Export und Suche. Große Listen werden paginiert oder extern gespeichert.

Datenart	In Prozessvariable?	Begründung	Beispiel
Korrelation und Status	Ja	Klein, prozesssteuernd und häufig benötigt	orderId, approvalState
Entscheidungseingaben	Ja, minimiert	Für Nachvollziehbarkeit und Regelaufwurf relevant	riskClass, amount
Vollständiges Domänenaggregat	Meist nein	Ändert sich unabhängig und bläht jede Aktualisierung auf	Komplette Bestellung mit Positionen
Dokument oder Bild	Nein	Binärdaten belasten Payload und Historie	PDF, Scan, Foto
Zugriffstoken und Secret	Nein	Sicherheits- und Lebenszyklusrisiko	OAuth Access Token
Temporäres Mappingergebnis	Lokal oder gar nicht	Soll globale Instanz nicht dauerhaft vergrößern	Connector-Zwischenergebnis

Datenart	In Prozessvariable?	Begründung	Beispiel
Audit-relevante Fachentscheidung	Ja oder extern mit Referenz	Muss nachweisbar und versioniert sein	Decision Result und Rule Version

7.3 Migrationsgefährliche Variablentypen in Camunda 7

Befund	Warum problematisch	Bereinigung vor der Migration
Java Serialized Object	Benötigt Klasse und kompatible Serialisierung; Sicherheitsrisiko	Explizites JSON-DTO oder externe ID
Spin JSON oder XML mit API-Aufrufen in Expressions	Ausdruck bindet sich an Spin und JUEL	Plain JSON plus FEEL oder Worker-Mapping
File Value und große Byte Arrays	Payload, Migration und Historie werden schwer	Objektspeicher plus URI, Hash, MIME-Type
Custom Object Value Serializer	Plattformspezifischer Code ohne C8-Pendant	Neutraler Datenvertrag
Variablenname mit Sondersemantik oder ungültiger Zielsyntax	Data Migrator und FEEL können Grenzen haben	Namenskonvention und Transformation
Unbegrenzte Listen oder Maps	Instanzgröße wächst mit Laufzeit	Aggregation extern, nur Zähler und Referenz im Prozess
Geheime Daten	Historisierung und breite Sichtbarkeit	Secret Manager und kurzlebige Referenz

7.4 Von JUEL zu FEEL

Camunda-7-Muster	FEEL-orientierte Form	Kommentar
<code>\${amount > 1000}</code>	<code>= amount > 1000</code>	Einfacher Datenvergleich lässt sich direkt übertragen
<code>\${customer.vip && order.total > 0}</code>	<code>= customer.vip and order.total > 0</code>	Operatoren und Nullfälle testen
<code>\${myBean.calculate(execution)}</code>	Kein direktes Pendant	Geschäftslogik in Worker oder Decision verschieben
<code>\${execution.getVariable("x")}</code>	<code>= x</code>	Engine-Kontextzugriff entfernen
<code>\${S(orderJson).prop("id").stringValue()}</code>	<code>= order.id</code>	Plain JSON und direkter Datenzugriff
<code>\${dateTime().plusDays(3)}</code>	FEEL date/time/duration oder Zeitdienst	Zeitzone und Kalenderlogik explizit testen
<code>\${empty items}</code>	<code>= items = null or count(items) = 0</code>	Exakte FEEL-Semantik und fehlende Variable prüfen

7.5 Datenvertrag als Schulungsartefakt

Feld	Pflichtinhalt
Name und fachliche Bedeutung	Eindeutige Beschreibung ohne Bezug auf Java-Klasse
JSON-Typ und Schema	String, Number, Boolean, Object, Array; erlaubte Struktur
Owner	Team oder System, das Semantik und Version verantwortet
Quelle und Lebenszyklus	Wann erzeugt, geändert und gelöscht?
Sichtbarkeit	Global, lokaler Scope, nur Worker, nur UI

Feld	Pflichtinhalt
Größenbudget	Normalwert, Warnschwelle, Maximum
Datenschutzklasse	Personenbezug, Geheimnis, Aufbewahrung und Löschung
Versionierung	Kompatible Erweiterung, Breaking Change und Transformationsregel

Quellenbasis: *Conceptual differences; variable documentation; Data Migrator variable transformations and limitations*

KAPITEL 8

Anwendungscode und Integrationen refaktorisieren

Das Ziel ist eine lose gekoppelte Orchestrierung. Geschäftscode bleibt testbar und wird über stabile Verträge an die Engine angebunden.

8.1 Vom JavaDelegate zum Worker

Camunda 7: typisches Delegate-Muster

```
public final class ChargeCardDelegate implements JavaDelegate {
    public void execute(DelegateExecution execution) {
        String orderId = (String) execution.getVariable("orderId");
        receiptRepository.save(paymentService.charge(orderId));
        execution.setVariable("paymentStatus", "PAID");
    }
}
```

Camunda 8: typisches Worker-Muster, verkürzt

```
@JobWorker(type = "charge-card")
public Map<String, Object> charge(ActivatedJob job) {
    String orderId = readRequiredString(job, "orderId");
    String key = "charge-card:" + orderId;
    Receipt receipt = paymentService.chargeIdempotently(orderId, key);
    return Map.of("paymentStatus", "PAID", "receiptId", receipt.id());
}
```

Der sichtbare Unterschied ist eine Annotation. Der entscheidende Unterschied liegt in der Fehler- und Transaktionssemantik: Der Worker muss Eingaben validieren, externe Seiteneffekte deduplizieren, technische Fehler klassifizieren und nur kleine Ergebnisvariablen zurückgeben.

8.2 Refactoring-Sequenz

1. Delegate auf Engine-API-Nutzung, Transaktionsannahmen, Variablenzugriff und Seiteneffekte analysieren.
2. Geschäftslogik in einen engineunabhängigen Application Service verschieben.
3. Ein- und Ausgabe als versionierten JSON-Vertrag definieren.
4. Idempotenzschlüssel und Fehlerklassifikation festlegen.
5. Worker-Adapter implementieren, der Vertrag validiert und den Application Service aufruft.
6. Unit Tests ohne Camunda schreiben, danach Worker-Vertrag und Prozessintegration testen.
7. Metriken, strukturierte Logs, Trace-Kontext und Alarmierung ergänzen.

8.3 Mapping wichtiger Camunda-7-Erweiterungspunkte

Camunda-7-Erweiterung	Typische Nutzung	Camunda-8-Zielmuster
JavaDelegate	Service Task Logik	Job Worker oder Connector
Delegate Expression	Spring Bean aus BPMN aufrufen	Jobtyp als Vertrag, Bean hinter Worker
ExecutionListener	Audit, Variablen, Integration, technische Hooks	Execution Listener Job Worker, Prozessmodell oder Event-Integration
TaskListener	Assignment, Validierung, Benachrichtigung	Camunda User Task Listener, Worker, UI oder Event
ProcessEnginePlugin	Globale Engine-Verhaltensänderung	Unterstützte Konfiguration, API-Gateway, Worker, Export oder Plattformservice

Camunda-7-Erweiterung	Typische Nutzung	Camunda-8-Zielmuster
BpmnParseListener	Modelle beim Deployment verändern	Build-time-Transformation, Element Templates, Modellierungsrichtlinie
HistoryEventHandler	Audit oder Data Warehouse	Unterstützter Export, API, Webhook oder Datenpipeline
Custom Incident Handler	Spezielle Fehlerreaktion	Standardincident plus Automation über API und Eventing
Cockpit Plugin	Eigene operative Oberfläche	Eigene Anwendung auf Public APIs oder unterstützte Erweiterung
Custom Variable Serializer	Domänentyp in Engine speichern	JSON-Schema und externer Fachspeicher
CommandInterceptor	Querschnittliches Verhalten	Service Boundary, API Policy, Client Interceptor oder Plattformbeobachtung

8.4 External Task zu Job Worker

Konzept	Camunda 7 External Task	Camunda 8 Job Worker	Migrationsprüfung
Routing	Topic Name	Job Type	Namenskonvention und Ownership
Akquisition	Fetch and Lock	Activate Jobs oder Client Worker	Max Jobs Active und Backpressure
Sperre	Lock Duration und Extend Lock	Job Timeout und Update Timeout	Laufzeitmessung und Heartbeat-Strategie
Abschluss	Complete External Task	Complete Job	Ergebnisvariablen klein halten
Fehler	Handle Failure, Retries, Retry Timeout	Fail Job, Retries, Retry Backoff	Backoff nicht nur konstant
Fachfehler	Handle BPMN Error	Throw BPMN Error	Error Code und Variablenvertrag
Variablen	Fetch Variables und Local Variables	Fetch Variables, result variables	Scope und Mapping testen
Priorität	External Task Priority	Aktivierungs- und Prozessmechanismen unterscheiden sich	Laststeuerung neu entwerfen

8.5 Connector oder eigener Worker?

Kriterium	Connector bevorzugen	Worker bevorzugen
Integration	Standardprotokoll oder verbreitetes SaaS-System	Domänenspezifische API oder komplexe Bibliothek
Logik	Mapping und überschaubare Konfiguration	Mehrstufige Regeln, Cache oder Fachtransaktion
Betrieb	Zentraler Connector Runtime Betrieb gewünscht	Unabhängige Skalierung und Deployment durch Produktteam
Security	Secrets über vorgesehenes Connector-Konzept	Spezielle Authentisierung oder Netzwerkzone
Wiederverwendung	Viele Prozesse nutzen dasselbe standardisierte Template	Spezifischer Servicevertrag

Kriterium	Connector bevorzugen	Worker bevorzugen
Testbarkeit	Connector- und Template-Tests reichen	Vollständige Unit-, Contract- und Integrationstests nötig

8.6 Keine Adapterfalle

Adapter als Übergang, nicht als Ziel

Ein generischer Worker, der alte JavaDelegate-Klassen mit simuliertem DelegateExecution aufruft, verkürzt den ersten Port. Er konserviert jedoch Engine-Abhängigkeiten, Transaktionsannahmen und unklare Variablenverträge. Für große Bestände kann er eine zeitlich begrenzte Brücke sein, muss aber ein Ablaufdatum und Refactoring-Backlog besitzen.

Quellenbasis: *Conceptual differences; migration journey; Camunda Client, worker and connector documentation*

KAPITEL 9

User Tasks, Formulare, Identity und Multi-Tenancy

Human Workflow verbindet Prozesszustand mit Benutzeroberfläche, Rollen, Datenschutz und organisatorischer Verantwortung. Eine reine BPMN-Konvertierung reicht deshalb nicht.

9.1 Camunda User Task als Zielstandard

Für neue Camunda-8-Modelle ist der Camunda User Task der vorgesehene Tasktyp für Task-Lifecycle und Query-Funktionen. Job-based User Tasks befinden sich im Abkündigungspfad für Task Management. Die XML-Bezeichnung kann historisch noch Zeebe User Task enthalten, bezeichnet aber denselben engine-nativen Tasktyp.

Aspekt	Zu klären
Erzeugung und Zustand	Welche Taskzustände und Übergänge nutzt UI oder Integration?
Assignment	Assignee, Candidate Users, Candidate Groups, Claim und Unclaim
Fälligkeit und Follow-up	Zeitzone, Eskalation, Sortierung und SLA
Formular	Camunda Form, externe URL oder eigene Anwendung
Listener	Welche Lifecycle-Ereignisse werden benötigt und sind im Zielrelease unterstützt?
Zugriff	Wer darf Tasks suchen, sehen, beanspruchen und abschließen?
Variablen	Welche Daten werden angezeigt, bearbeitet oder lokal gehalten?
Audit	Welche Benutzeraktion muss mit Zeit, Identität und Datenänderung nachweisbar sein?

9.2 Formularmigration

Ausgang in Camunda 7	Zieloption	Migrationsthema
Embedded Task Form mit HTML und JavaScript	Eigene Webanwendung oder neu erstelltes Camunda Form	Direkte Portierung ist nicht vorgesehen; API, Authentisierung und Validierung neu bauen
Generated Task Form	Camunda Form	Feldtypen, Validierung und Layout nachbauen
Camunda Form	Konvertiertes oder neu erstelltes Camunda Form	Komponentenkompatibilität und Form Binding prüfen
External Task Form URL	Eigene Anwendung	Taskzugriff und Callback über C8 APIs
Start Form	Eigene Startanwendung oder unterstützte Formfunktion	Öffentlicher Start, Authentisierung und Deprecations beachten
Formularlogik mit JUEL oder Beans	FEEL, UI-Logik oder Backend-Service	Verantwortung und Sicherheitsgrenze trennen

Warum UI früh migriert werden sollte

Formulare berühren Task-APIs, Variablen, Rollen, Validierung und Deployment. Werden sie erst am Ende betrachtet, treffen mehrere ungeklärte Systemgrenzen gleichzeitig aufeinander und blockieren den Cutover.

9.3 Identity- und Berechtigungsmapping

Camunda-7-Frage	Camunda-8-Entscheidung	Nachweis
Woher stammen Benutzer und Gruppen?	OIDC-Provider, Claims und Synchronisationsmodell	Login- und Claim-Test
Wie werden Kandidatengruppen benannt?	Stabile fachliche Rollen statt technischer Verzeichnisnamen	Rollenmatrix
Wer darf Betriebsdaten sehen?	Cluster-, Operate- und Task-Zugriffe nach Aufgaben trennen	Negativtests
Wie werden technische Clients authentisiert?	Client Credentials, Secrets und Rotation	Automatisierter Rotationstest
Wie werden Mandanten getrennt?	Tenant-Konzept, Deployment, Identity und Datenisolation des Zielreleases	Isolations- und Penetrationstest
Welche personenbezogenen Daten liegen in Variablen?	Minimierung, Maskierung, Retention und Löschung	Datenschutzfolge und Cleanup-Test

9.4 Multi-Tenancy nicht mechanisch übertragen

Die Existenz einer Tenant ID in beiden Produktlinien bedeutet nicht, dass Betriebs- und Isolationsmodell gleich sind. Zu entscheiden ist, ob Mandanten logisch in einer Plattform, durch getrennte Cluster, durch getrennte Umgebungen oder durch eine Kombination isoliert werden. Maßgeblich sind Sicherheitsanforderung, Datenresidenz, Releaseunabhängigkeit, Last und Betriebsaufwand.

Isolationsziel	Mögliche Maßnahme	Trade-off
Berechtigungstrennung	Tenant-aware Identity und API Authorization	Fehlkonfiguration kann Daten sichtbar machen
Datenresidenz	Getrennte Regionen oder Cluster	Mehr Betriebsaufwand
Releaseunabhängigkeit	Getrennte Cluster oder Umgebungen	Mehr Kosten und Automatisierung nötig
Lastisolation	Getrennte Cluster, Partitionierung oder Quoten	Kapazität kann weniger effizient genutzt werden
Kryptografische Isolation	Getrennte Schlüssel und gegebenenfalls Infrastruktur	Komplexere Wiederherstellung

Quellenbasis: Camunda User Task, forms, Identity/Admin, authorization and multi-tenancy documentation; 8.8 deprecations

KAPITEL 10

APIs, Tests, Versionierung und Deployment

Eine Migration ist nur beherrschbar, wenn APIs stabil gekapselt, Tests schichtweise aufgebaut und Deployments reproduzierbar sind.

10.1 API-Mapping nach Use Case

Use Case	Camunda 7	Camunda 8	Migrationshinweis
Ressourcen deployen	RepositoryService oder REST Deployment	Deploy Resources über Client oder REST	BPMN, DMN und Forms gemeinsam versionieren
Prozess starten	RuntimeService oder REST	Create Process Instance	Business Key zu Business ID und Variablenvertrag prüfen
Message korrelieren	RuntimeService correlateMessage	Publish Message	TTL, Correlation Key und Message ID
Variablen ändern	RuntimeService setVariables	Commands und APIs je Kontext	Race Conditions und Scope beachten
Tasks suchen	TaskService oder REST	Orchestration Cluster API oder vorgesehene Task APIs	Nicht auf deprecated v1 APIs neu aufbauen
Task abschließen	TaskService complete	Complete User Task	Taskzustand, Validierung und Authentisierung
Instanz abbrechen	deleteProcessInstance	Cancel Process Instance	Kompensation und externe Seiteneffekte bleiben fachlich
Instanz ändern	Process Instance Modification	Unterstützte Process Instance Modification Funktionen des Zielreleases	Use Case und Limit prüfen
Historie suchen	HistoryService oder REST	Such- und Analyse-APIs auf Sekundärspeicher	Eventual Consistency und Retention
Incident behandeln	ManagementService oder REST	Incident APIs und Retry-Commands	Runbook und Berechtigung

10.2 API-Kapselung

Architekturregel

Fachsysteme sollten keine Camunda-API-Details in ihre Domänenlogik streuen. Ein Process Gateway oder Application Service kapselt Start, Message, Task und Query. Dadurch können API-Version, Authentisierung, Retry und Observability zentral geändert werden.

Schicht	Verantwortung	Nicht dort
Domäne	Fachregeln und Invarianten	Camunda Client, Task IDs, BPMN-Schlüssel
Application Service	Use Case und Transaktionsgrenze des Fachsystems	HTTP- und Authentisierungsdetails
Process Gateway	Camunda Commands, Queries, Mapping, Retry und Telemetrie	Fachliche Entscheidungen
Transport	REST, Client, Credentials und Timeouts	Prozesslogik

10.3 Testpyramide für die Migration

Ebene	Ziel	Beispiele	Geschwindigkeit
Unit Test	Fachlogik ohne Engine	Application Service, Mapper, Idempotenz	Sehr schnell
Contract Test	JSON, API und Connectorvertrag	Schema, Fehlermapping, Versionstoleranz	Schnell
Process Model Test	Tokenfluss, Bedingungen, Timer, Fehlerpfade	Camunda Process Test	Mittel
Worker Integration Test	Client, Aktivierung, Complete, Fail und BPMN Error	Testcluster oder Containerumgebung	Mittel
End-to-End-Test	UI, Identity, Prozess, Worker und Downstream	Repräsentativer Geschäftsvorgang	Langsam
Resilienztest	Ausfall nach Fach-Commit, Timeout, Duplikat, Backpressure	Fault Injection	Gezielt
Migrationstest	Konvertierung, Datenmigration und Cutover	Produktionsnahe Kopie und Zählvergleich	Aufwendig

10.4 Pflichtenzenarien für Worker

Worker-Testcheckliste

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Wird derselbe Job zweimal verarbeitet, ohne den Seiteneffekt zu verdoppeln?	Idempotenztest mit identischem Schlüssel	
2	Was geschieht nach Fach-Commit, aber vor Complete?	Fault Injection und Wiederholung	
3	Werden temporäre und permanente Fehler unterschiedlich behandelt?	Retry- und Incident-Test	
4	Wird ein BPMN Error mit korrektem Code und Variablen ausgelöst?	Fachfehlerpfad	
5	Sind Timeouts länger als normale Laufzeit, aber kurz genug für Wiederanlauf?	Laufzeitverteilung und Timeout-Test	
6	Begrenzt der Worker parallele Jobs passend zum Downstream?	Last- und Backpressure-Test	
7	Sind Logs über Process Instance Key, Job Key und fachliche ID korrelierbar?	Log- und Trace-Nachweis	

10.5 Versionierung und Deployment

Artefakt	Versionsregel	Kompatibilitätsregel
BPMN	Immutable Deployment; neue Version statt Überschreiben	Aktive Instanzen bleiben auf ihrer Version, sofern nicht migriert

Artefakt	Versionsregel	Kompatibilitätsregel
DMN	Decision ID und Binding bewusst wählen	Neue Regeln dürfen alte Prozessversionen nicht unbeabsichtigt ändern
Form	Form ID und Binding mit Prozessrelease koordinieren	Feldänderungen mit Variablenvertrag kompatibel halten
Worker	Jobtyp stabil, Implementierung rollbar	Während Rolling Deployment beide Prozessversionen bedienen können
Connector Template	Template-Version und Connector Runtime abstimmen	Modelle dürfen keine entfernten Properties erwarten
API Client	Public API und unterstützte SDK-Version verwenden	Deprecation-Warnungen in CI sichtbar machen

10.6 Release-Gate

1. Alle Artefakte sind reproduzierbar gebaut und signiert oder versioniert.
2. Modelle wurden linted, deployed und gegen das Zielrelease getestet.
3. Worker unterstützen alte und neue Modellversion während des Rollouts.
4. API- und SDK-Deprecations wurden geprüft und dokumentiert.
5. Rollback unterscheidet Anwendungscode, Modellversion und Plattformzustand.
6. Dashboards und Alerts sind vor Produktionsverkehr aktiv.
7. Ein fachlicher Smoke Test bestätigt Start, Wartezustand, Task, Message und Abschluss.

Quellenbasis: Camunda public API definition; Orchestration Cluster REST API; Camunda Process Test; deployment and versioning documentation

KAPITEL 11

Betrieb, Plattform und Governance

Camunda 8 verlagert den betrieblichen Schwerpunkt von einer Java-Engine mit relationaler Datenbank zu einem verteilten Orchestrierungsdienst mit mehreren skalierbaren Komponenten.

11.1 Betriebsmodell zuerst entscheiden

Kriterium	Camunda 8 SaaS	Camunda 8 Self-Managed
Plattformbetrieb	Camunda betreibt Kernplattform und Upgrades gemäß Service	Eigene Organisation betreibt Plattform, Upgrades, Backup und Kapazität
Infrastrukturwahl	Vorgegebene Regionen und Serviceoptionen	Eigene Cloud, Kubernetes oder unterstützte Installationsform
Upgrade-Kontrolle	Release- und Maintenance-Prozess des Dienstes	Eigene Terminierung innerhalb unterstützter Upgradepfade
Datenzugriff	Über bereitgestellte APIs und Funktionen	Zusätzlicher Infrastrukturzugriff, aber nur dokumentierte Schnittstellen nutzen
History Data Migrator	Nicht direkt in SaaS-Datenbank möglich	Mit RDBMS-Sekundärspeicher und Voraussetzungen möglich
Compliance	Vertrag, Region, Zertifikate und Servicegrenzen prüfen	Eigene Verantwortung für Hardening, Backup, Residenz und Nachweise
Kostenmodell	Verbrauchs- oder Vertragsmodell	Lizenz plus Infrastruktur und Betriebsteam
Time to Platform	Schnellerer Start möglich	Mehr Aufbau, dafür tiefere Infrastrukturkontrolle

Entscheidungsursache

SaaS oder Self-Managed ist keine reine Hostingfrage. Die Wahl verändert Verantwortlichkeit für Upgrade, Datenzugriff, Backup, Netzwerk, Identity, Audit und Data-Migrator-Möglichkeiten.

11.2 Komponentenlandkarte

Funktion	Typische Komponente oder Schnittstelle	Betriebsaufgabe
Orchestrierung	Orchestration Cluster	Partitionen, Replikation, Ressourcen und Verfügbarkeit
Clientzugriff	REST und Client-Endpunkte	TLS, Authentisierung, Routing, Rate und Timeout
Operative Steuerung	Operate-Funktionen und Orchestration Cluster APIs	Incidents, Instanzsicht und Berechtigungen
Human Tasks	Tasklist-Funktionen und User Task APIs	Zugriff, Task-Lifecycle und UI-Verfügbarkeit
Identity und Administration	Admin- und Identity-Funktionen	OIDC, Claims, Rollen, technische Clients
Modellierung	Desktop Modeler oder Web Modeler	Templates, Versionsführung und Deploymentfreigabe

Funktion	Typische Komponente oder Schnittstelle	Betriebsaufgabe
Connectoren	Connector Runtime	Secrets, Netzwerk, Skalierung und Patchen
Sekundärspeicher	Elasticsearch, OpenSearch oder RDBMS je Zielarchitektur und Release	Kapazität, Retention, Backup, Reindex und Performance
Worker	Eigene Services	Deployment, Autoscaling, Idempotenz und Downstream-Schutz
Analyse	Optimize oder externe BI je Lizenz und Architektur	Datenmodell, Retention und Zugriffsrechte

11.3 Primär- und Sekundärzustand

Der Orchestrierungszustand wird im verteilten Runtime-Kern geführt. Operative und historische Suchsichten werden in einen Sekundärspeicher exportiert. Seit Camunda 8.9 kann ein relationaler Sekundärspeicher Teil unterstützter Self-Managed-Architekturen sein. Das macht die relationale Datenbank jedoch nicht zur Runtime-Engine wie in Camunda 7.

Frage	Primärer Runtime-Zustand	Sekundäre Sicht
Zweck	Korrekte Prozessausführung	Suche, Betrieb, Historie und Analyse
Konsistenz	Command-Verarbeitung im Cluster	Kann zeitversetzt aktualisiert sein
Skalierung	Partitionen und Replikation	Index-, Tabellen- und Query-Kapazität
Backup	Cluster-spezifische konsistente Sicherung	Speicherspezifische Sicherung und Wiederaufbau
Integration	Unterstützte Commands und Ereignisse	Öffentliche Queries und Exportpfade
Direkter Zugriff	Nicht als Anwendungsdatenbank verwenden	Auch hier keine Bindung an interne Schemas

11.4 Kapazitätsmodell

Treiber	Messgröße	Typische Wirkung
Prozessstarts	Starts pro Sekunde und Burst	Command-Last und Exportvolumen
Aktive Instanzen	Anzahl, Alter und Wartezustände	Runtime-Zustand und Timer
Jobs	Jobs pro Sekunde, Laufzeit und Retryrate	Workerbedarf und Backpressure
Variablen	Bytes pro Instanz und Änderungsrate	Netzwerk, Log, Export und Speicher
User Tasks	Offene Tasks, Suchlast, Updates	Task-Query und UI-Last
Historie	Events pro Tag und Retention	Sekundärspeicher und Cleanup
Incidents	Anzahl, Alter und Wiederholungen	Operative Last und Speicher
Replikation	Replication Factor und Partitionen	CPU, Netzwerk und Speichermultiplikator
Downstream-Limits	Rate, Latenz und Fehlerrate	Workerparallelität und Queueaufbau

Lasttestregel

Ein Lasttest mit kleinen Variablen und fehlerfreien Downstream-Systemen misst nur den günstigsten Fall. Repräsentativ sind reale Payloadgrößen, Timerbursts, Retryspitzen, langsame Worker und parallele Suchabfragen.

11.5 Monitoring und Service Level Indicators

Bereich	Beispielindikatoren	Alarmursache
Cluster	Command-Latenz, Backpressure, Partition Health, Replikation	Verfügbarkeits- oder Kapazitätsverlust
APIs	Fehlerrate, Latenz, Authentisierungsfehler, Rate Limits	Client- oder Plattformstörung
Worker	Aktive Jobs, Bearbeitungszeit, Retries, Timeouts, Deduplizierungen	Downstream oder Implementierungsproblem
Sekundärspeicher	Exportlag, Query-Latenz, Speicherauslastung, Cleanup	Operative Sichten veralten oder fallen aus
Prozess	Start- und Abschlussrate, Durchlaufzeit, Warteschlangen, Incidents	Fachlicher oder technischer Engpass
User Tasks	Alter, Fälligkeit, Claim- und Complete-Fehler	Organisatorischer Rückstau
Migration	Zählabweichung, nicht migrierbare Instanzen, Fehlerklassen	Cutover-Risiko

11.6 Backup, Restore und Disaster Recovery

1. Recovery Point Objective und Recovery Time Objective je Geschäftsprozess festlegen.
2. Alle zustandsführenden Komponenten und Abhängigkeiten inventarisieren. Ein einzelnes Datenbankbackup ist nicht automatisch ein Plattformbackup.
3. Konsistenzanforderungen der offiziellen Backup-Dokumentation des Zielreleases umsetzen.
4. Restore in eine isolierte Umgebung automatisieren und regelmäßig testen.
5. Nach Restore fachliche Smoke Tests, Instanzzählung, Workeraktivierung, Taskzugriff und Suchsicht prüfen.
6. Secrets, Zertifikate, Identity-Konfiguration und DNS als Teil des Wiederanlaufs behandeln.
7. Rollback eines Upgrades nicht mit Disaster Recovery verwechseln. Beide benötigen eigene Runbooks.

11.7 Security und Governance

Kontrolle	Umsetzung
Public API only	Zugriffe auf dokumentierte öffentliche APIs beschränken und in Architekturtests prüfen
Least Privilege	Benutzer, technische Clients und Worker nur mit erforderlichen Rechten ausstatten
Secret Management	Keine Secrets in BPMN, Variablen, Images oder Git; Rotation automatisieren
Supply Chain	Images, SDKs, Connectoren und Charts signieren, scannen und

Kontrolle	Umsetzung
	versionieren
Datenminimierung	Personenbezogene Daten in Variablen reduzieren; Retention und Löschung testen
Netzwerk	Egress, Ingress, interne Endpunkte und Connectorziele begrenzen
Audit	Administrative und fachliche Änderungen mit Identität und Zeit nachweisen
Supported Environments	JDK, Datenbanken, Kubernetes, Browser und Suchspeicher je Release prüfen
Lizenz	Self-Managed-Produktion benötigt seit Camunda 8.6 ein passendes Produktionsrecht

Quellenbasis: *Camunda 8 architecture, storage, supported environments, licensing, security, backup and monitoring documentation*

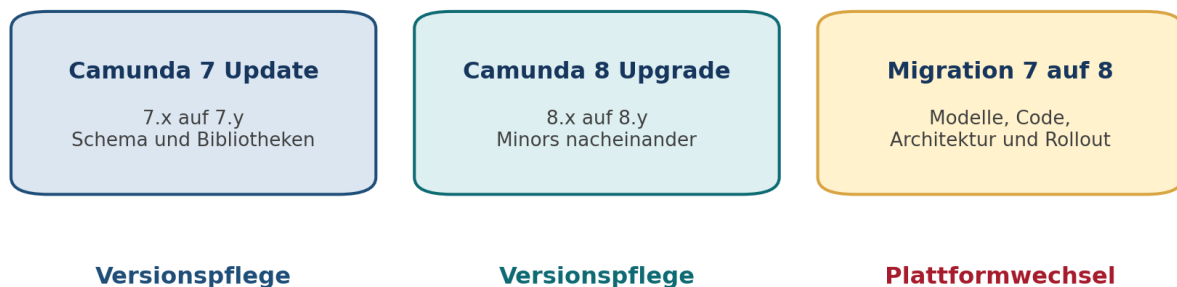
KAPITEL 12

Updates, Upgrades und technische Wartung

Versionspflege folgt innerhalb jeder Produktlinie anderen Regeln. Das Kapitel trennt Camunda-7-Update, Camunda-8-Upgrade und die Migration von 7 auf 8.

12.1 Begriffe sauber trennen

Drei unterschiedliche Vorhaben



Ein Plattformwechsel kann nicht durch den Austausch einer Bibliothek erledigt werden.

Abbildung 5: Update, Upgrade und Migration im Vergleich.

Vorhaben	Beispiel	Hauptänderung	Rollback-Grundlage
Camunda 7 Patch Update	7.24.2 auf 7.24.5	Bibliotheken und Fehlerkorrekturen innerhalb eines Minor Releases	Artefaktrollback nach Kompatibilitätsprüfung
Camunda 7 Minor Update	7.22 auf 7.24	Zwischenversionen, Datenbankschema und Konfiguration	DB-Backup und versionsspezifischer Plan
Camunda 8 Patch Upgrade	8.9.x auf neueren 8.9-Patch	Komponentenimages oder Binaries	Releaseleitfaden und Backup
Camunda 8 Minor Upgrade	8.8 auf 8.9	Plattformkomponenten, Konfiguration, Daten und APIs	Restore aus vollständigem Backup; Downgrade nicht voraussetzen
Migration 7 auf 8	7.24-Lösung auf 8.9-Ziel	Architektur, Modelle, Code, Betrieb und Daten	Rollout- und Rückfallstrategie auf Lösungsniveau

12.2 Camunda 7 Patch Update

1. Release Notes, bekannte Probleme und Sicherheitsinformationen aller übersprungenen Patches prüfen.
2. Unterstützte JDK-, Datenbank-, Application-Server- und Frameworkversionen abgleichen.
3. Alle Camunda-Bibliotheken konsistent auf denselben Patchstand bringen.
4. Anwendungs- und Prozessregressionstests durchführen. Besonders Jobs, Authentisierung, REST und Plugins prüfen.
5. Bei Clusterbetrieb Kompatibilität der gemischten Patchstände und Rolloutreihenfolge dokumentieren.

6. Nach Deployment Engineversion, Schema, Job Executor, Cockpit und Tasklist verifizieren.

Patch ist nicht risikofrei

Ein Patch hält das Datenbankschema innerhalb des Minor Releases grundsätzlich kompatibel, kann aber Verhalten von Bibliotheken, Abhängigkeiten und Fehlerkorrekturen ändern. Deshalb bleiben Regressionstest und Rollback erforderlich.

12.3 Camunda 7 Minor Update

Phase	Pflichtschritte	Typischer Fehler
Analyse	Alle Update Guides zwischen Ausgang und Ziel lesen	Nur Zielversion prüfen und Zwischenänderungen übersehen
Umgebung	Supported Environments für Zielversion herstellen	JDK oder Datenbank gleichzeitig ungeplant ändern
Schema	Jedes sequenzielle Datenbank-Update-Skript anwenden	Skripte sind nicht kumulativ; Zwischenversion auslassen
Bibliotheken	Abhängigkeiten, Webapps und Clients aktualisieren	Gemischte Versionen oder transitive Altbibliotheken
Custom Code	Plugins, Listener, Serializers und interne API-Nutzung prüfen	Kompilierung reicht, Laufzeitpfad bleibt ungetestet
Test	Prozess-, Job-, REST-, UI- und Migrationsregression	Nur Happy Path testen
Rollout	Downtime oder Rolling Update gemäß Leitfaden	Neue Features während gemischtem Cluster nutzen
Verifikation	Schema, Engine, Jobs, Historie und Webapps prüfen	Nur HTTP-Healthcheck verwenden

Wichtige Ursache

Camunda-7-Datenbankskripte für Minor Updates sind nicht kumulativ. Die Bibliothek kann größere Versionssprünge technisch erlauben, das Datenbankschema benötigt dennoch jedes Zwischenupdate in korrekter Reihenfolge.

12.4 Rolling Update in Camunda 7

1. Ausgangsminor auf den aktuellen Patchstand bringen.
2. Nur auf das direkt folgende Minor Release rollen. Mehrere Minor Releases werden nacheinander behandelt.
3. Datenbankschema mit den als online geeigneten Schritten vorbereiten; Sperren und Laufzeit messen.
4. Während gemischter Engineversionen keine Features verwenden, die der alte Knoten nicht versteht.
5. Knoten einzeln ersetzen und Engine-, Job- sowie Deploymentverhalten beobachten.
6. Nach vollständigem Rollout verbleibende Schema- oder Konfigurationsschritte abschließen.

Rolling Update reduziert Downtime, beseitigt aber nicht das Risiko von DDL-Locks, gemischter Semantik oder inkompatiblen Erweiterungen. Eine produktionsähnliche Probe mit Datenvolumen ist erforderlich.

12.5 Camunda 8 Minor Upgrade, Grundregeln

1. Minor Releases sequenziell aktualisieren. Für den Referenzpfad auf 8.9 ist zuerst 8.8 erforderlich.
2. Vor dem Minor Upgrade den neuesten unterstützten Patch der Ausgangsversion einsetzen.
3. Release Announcements, Breaking Changes, Deprecations, Supported Environments und Komponentenleitfäden lesen.
4. Vollständiges, getestetes Backup erstellen. Ein Downgrade auf das vorherige Minor Release ist nicht als normaler Rollbackpfad vorgesehen.
5. Konfiguration nicht blind ersetzen. Neue Standardwerte und umbenannte Properties kontrolliert zusammenführen.
6. SDKs, Connector Runtime, Helm Charts, Images, Identity und Sekundärspeicher als zusammenhängende Kompatibilitätsmatrix behandeln.
7. Nach Upgrade sowohl Commands als auch sekundäre Suchsichten und User Tasks verifizieren.

12.6 Manuelles Upgrade von Camunda 8

Schritt	Handlung	Nachweis
1. Planen	Pfad, Zielpatch, Komponenten, Downtime und Verantwortliche festlegen	Freigegebener Runbookstand
2. Sichern	Runtime- und Sekundärdaten, Konfiguration, Secrets und Artefakte sichern	Restore-Test oder aktueller Nachweis
3. Stoppen	Camunda-Komponenten kontrolliert beenden	Keine laufenden Schreibvorgänge
4. Software wechseln	Alte Bibliotheken oder Distribution entfernen, neue Version bereitstellen	Keine Altartefakte im Klassenpfad
5. Konfiguration mergen	Eigene Werte in neue Referenzkonfiguration übertragen	Diff und Peer Review
6. Starten	Komponenten in dokumentierter Reihenfolge starten	Health, Logs, Partitionen und Export
7. Prüfen	Deployment, Start, Worker, Message, Task, Incident und Suche testen	Abgezeichneter Smoke Test
8. Beobachten	Latenz, Backpressure, Fehler und Speicher prüfen	Stabilitätsfenster und Abbruchkriterien

12.7 Helm-basiertes Upgrade

Upgrade-Checkliste für Kubernetes und Helm

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Ist der Helm-Chart-Pfad für Ausgang und Ziel dokumentiert?	Chart- und App-Versionen	
2	Sind alle Values gegen die neue Referenz verglichen?	Versionierter Values-Diff	
3	Sind entfernte und umbenannte Properties bereinigt?	Config-Lint und Release Announcement	
4	Sind Images, Registry, Pull	Preflight in Testcluster	

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
	Secrets und Security Context unterstützt?		
5	Sind Storage Classes, Volumes und Backup kompatibel?	Restore-Probe	
6	Sind PodDisruptionBudget, Affinity und Ressourcen passend?	Resilienz- und Kapazitätstest	
7	Sind CRDs, Operatoren und Ingress-Abhängigkeiten berücksichtigt?	Plattformfreigabe	
8	Ist ein klarer Restore-basierter Rückfallweg vorhanden?	Durchgespieltes Runbook	

12.8 Gemeinsame Upgrade-Antipatterns

Antipattern	Ursache des Risikos	Bessere Praxis
Mehrere Minor Releases auf einmal überspringen	Versionen enthalten sequenzielle Daten- und Konfigurationsänderungen	Jeden unterstützten Zwischenschritt einzeln testen
Nur Anwendungscode testen	Betriebsdaten, Historie, Identity und Sekundärspeicher ändern sich ebenfalls	Plattform-Smoke-Test und fachliche Regression
Backup ohne Restore-Test	Sicherungsdatei beweist keine Wiederherstellbarkeit	Automatisierter Restore in isolierter Umgebung
Neue Defaults ungeprüft übernehmen	Sicherheit, Ressourcen oder Verhalten können sich ändern	Konfigurationsdiff mit Entscheidung je Property
Rollback als Image-Downgrade planen	Datenformat kann bereits migriert sein	Offiziellen Pfad nutzen, häufig Restore aus Backup
Deprecations ignorieren	Nächstes Minor entfernt Integrationspfad	Deprecation-Budget je Release

Quellenbasis: Camunda 7 patch, minor and rolling update guides; Camunda 8 upgrade to 8.9, prepare, manual and Helm upgrade guides

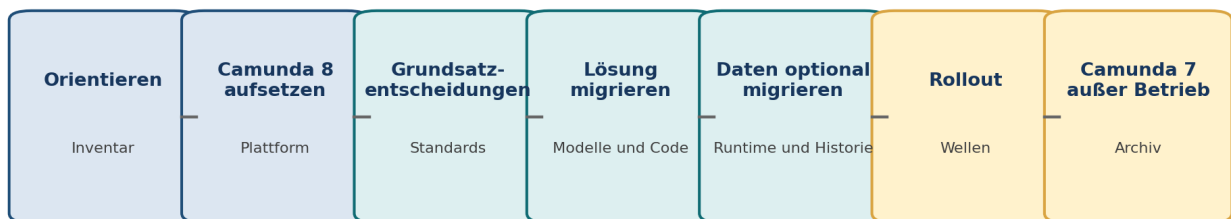
KAPITEL 13

Migration von Camunda 7 zu Camunda 8

Die Migration ist ein Transformationsprogramm mit Architektur-, Lösungs-, Daten- und Betriebsarbeit. Eine kontrollierte Reise reduziert gleichzeitige Änderungen und schafft messbare Gates.

13.1 Die Migrationsreise

Migrationsreise in sieben kontrollierten Etappen



Gate zwischen den Etappen: Architekturentscheidung, Testnachweis, Betriebsfreigabe und Rollback-Plan

Abbildung 6: Sieben Etappen der Migration.

Etappe	Kernartefakte	Exit-Kriterium
1. Orientieren	Portfolioinventar, Ziele, Business Case, Supportlage	Scope und Priorisierung beschlossen
2. Camunda 8 aufsetzen	Zielplattform, Landing Zone, Identity, CI/CD, Observability	Produktionsnaher Plattform-Smoke-test
3. Grundsatzentscheidungen	Workerstandard, Datenvertrag, UI, Messaging, Mandanten, Audit	Architecture Decision Records freigegeben
4. Lösung migrieren	Konvertierte Modelle, refaktoriierter Code, Tests, Forms	Fachliche und technische Abnahme
5. Daten optional migrieren	Instanzstrategie, Toolkonfiguration, Transformationen, Zählvergleich	Testmigration reproduzierbar
6. Rollout	Waveplan, Cutover, Parallelbetrieb, Supportmodell	Kohorte stabil und KPIs innerhalb Grenzen
7. Camunda 7 abschalten	Archiv, Retention, Audit, Vertrags- und Infrastrukturabbau	Keine produktiven Abhängigkeiten mehr

13.2 Portfolioinventar

Migrationsinventar

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Welche Prozessdefinitionen und Versionen existieren?	BPMN-Inventar mit Start- und Abschlusszahlen	
2	Welche JavaDelegates, External	Code- und Deploymentinventar	

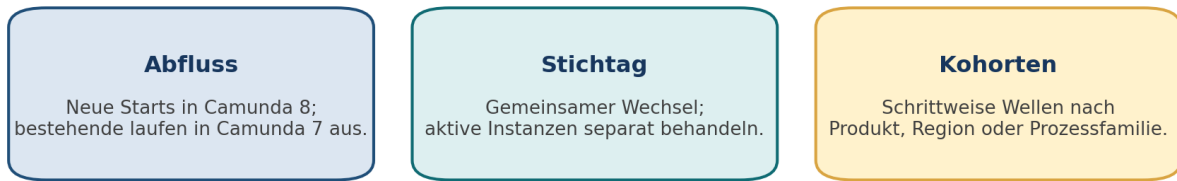
Nr.	Prüffrage	Nachweis oder Ergebnis	Status
	Tasks, Listener und Plugins werden genutzt?		
3	Welche Variablentypen und Größen treten auf?	Statistische Auswertung und Stichprobe	
4	Welche Formulare, Task-UIs und öffentlichen Startpunkte existieren?	UI-Katalog	
5	Welche aktiven Instanzen gibt es und wie alt sind sie?	Laufzeitverteilung und Wartezustände	
6	Welche direkten Datenbankzugriffe und Reports bestehen?	SQL-, BI- und ETL-Katalog	
7	Welche Authentisierungs-, Gruppen- und Mandantenmodelle gelten?	Identity- und Rollenmatrix	
8	Welche Compliance-, Retention- und Auditpflichten gelten?	Kontrollkatalog	
9	Welche Prozesse werden ohnehin abgelöst oder selten geändert?	Portfolioentscheidung	

13.3 Modernisierungsoptionen pro Prozess

Option	Wann sinnvoll	Konsequenz
Stilllegen	Prozess wird fachlich nicht mehr benötigt	Daten archivieren, Startkanäle schließen
Ersetzen	Standardsoftware oder anderer Dienst übernimmt Funktion	Keine technische Portierung, aber Schnittstellenmigration
Neu entwerfen	CMMN, starke Enginekopplung oder fachliche Neuausrichtung	Höherer initialer Aufwand, bessere Zielarchitektur
Refaktorisieren und migrieren	Fachmodell bleibt, technische Muster ändern sich	Typischer Brownfield-Pfad
Nahezu übernehmen	Einfaches BPMN, External Tasks, JSON, geringe UI-Kopplung	Konverter plus gezielte Anpassung
Zeitweise in C7 belassen	Abhängigkeit oder lange Instanz verhindert sofortige Migration	Parallelbetrieb mit Enddatum und Risikoakzeptanz

13.4 Rollout-Strategien

Rollout-Strategien für Prozesse und Instanzen



Komplexität	niedrig	hoch	mittel
Doppelbetrieb	hoch	niedrig	mittel
Lange Instanzen	bedingt	nur mit Datenstrategie	gut steuerbar
Rollback	einfacher	schwierig	kohortenweise

Abbildung 7: Abfluss, Stichtag und Kohorten.

Strategie	Vorteil	Risiko	Geeignet wenn
Abfluss oder Drain	Aktive Instanzen bleiben unverändert; geringer Datenmigrationsaufwand	Langer Parallelbetrieb und doppelte Betriebsprozesse	Instanzen kurz sind oder natürlich enden
Stichtag oder Big Bang	Kurze Parallelphase und klares Zielbild	Viele gleichzeitige Änderungen und schwerer Rückfall	Portfolio klein und Ausfallfenster akzeptabel ist
Kohorten	Risiko wird pro Region, Produkt oder Prozessfamilie begrenzt	Routing und temporäre Doppelstrukturen	Portfolio groß und schrittweise lernfähig ist
Hybrid	Je Prozess passende Methode	Governance und Transparenz werden komplexer	Instanzlaufzeiten und Kritikalität stark variieren

13.5 Entscheidung für laufende Instanzen

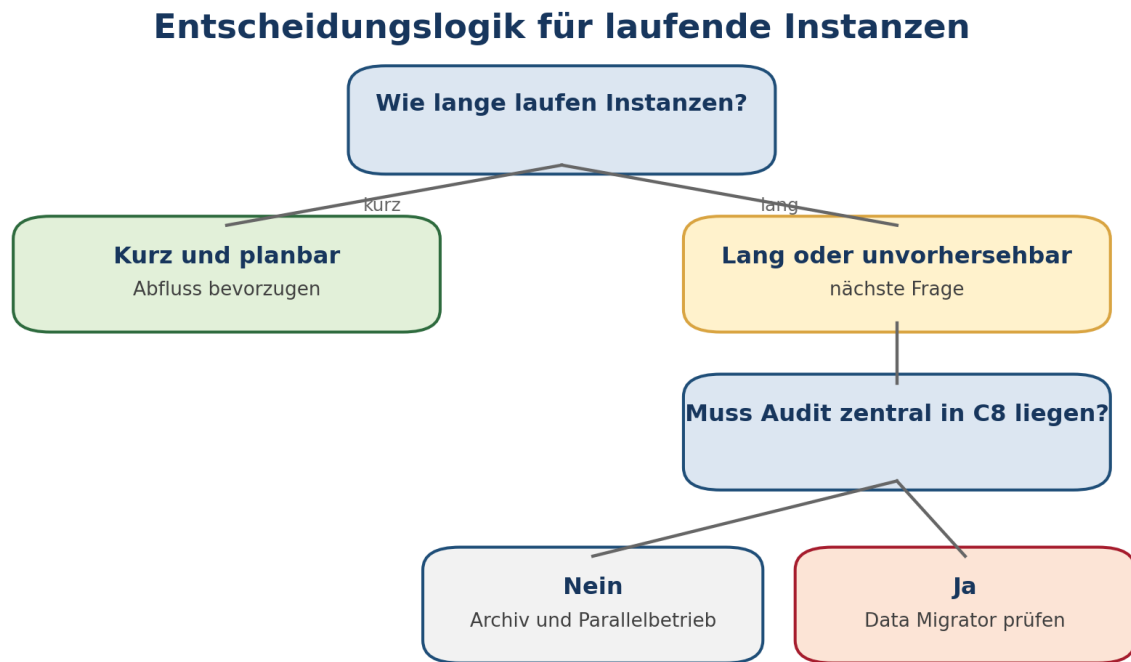


Abbildung 8: Vereinfachte Entscheidungslogik für laufende Instanzen.

Frage	Warum sie ursächlich wichtig ist
Wie lange laufen Instanzen?	Bestimmt Dauer und Kosten des Parallelbetriebs
In welchen Wartezuständen stehen sie?	Einige Zustände sind leichter kontrolliert neu zu starten als aktive Multi-Instance- oder Eventkonstruktionen
Welche externen Seiteneffekte sind bereits erfolgt?	Ein Neustart kann Zahlungen, Benachrichtigungen oder Buchungen verdoppeln
Muss vollständige Historie in Camunda 8 recherchierbar sein?	Entscheidet über Archiv statt History Migration
Sind Modell und Variablen durch Data Migrator unterstützt?	Toolgrenzen können Datenmigration ausschließen
Kann Camunda 7 bis zum Abfluss sicher betrieben werden?	Support, Infrastruktur und Personal begrenzen die Option

13.6 Parallelbetrieb als eigenes Produkt

Fähigkeit	Erforderliche Regel
Start-Routing	Eindeutig bestimmen, welche Plattform neue Instanzen startet
Message-Routing	Nachricht anhand Registry oder Fachzustand an richtige Plattform senden
Task-Suche	Benutzer sieht Aufgaben beider Plattformen oder erhält klare Trennung
Monitoring	Gemeinsames Lagebild mit Plattformkennzeichnung
Support	Incident-Routing und Runbooks für beide Plattformen
Deployment	Änderungen in C7 und C8 kontrollieren; keine divergierenden Fachregeln

Fähigkeit	Erforderliche Regel
Audit	Instanz und Historie plattformübergreifend auffindbar
Enddatum	Abschaltkriterien und maximale Parallelzeit festlegen

Warum Parallelbetrieb oft unterschätzt wird

Nicht zwei Engines verursachen den größten Aufwand, sondern das korrekte Routing von Starts, Nachrichten, Aufgaben, Supportfällen und Auditnachweisen. Ohne zentrale Instanzzuordnung können Ereignisse an die falsche Plattform gelangen.

13.7 Cutover-Prinzipien

1. Technische Migration und fachliche Produktänderung nur dann kombinieren, wenn Nutzen und Testbudget das Zusatzrisiko rechtfertigen.
2. Produktionsdatenmigration mehrfach mit anonymisierter oder geschützter Kopie proben.
3. Freeze-Fenster, letzte Datenänderung und Startstopp exakt definieren.
4. Rückfallkriterien vor Beginn quantifizieren, zum Beispiel Zählabweichung, Fehlerrate oder nicht erreichbare Tasks.
5. Kommunikation an Fachbereiche, Betrieb, Support und Audit als Teil des Runbooks behandeln.
6. Nach Cutover technische und fachliche Stabilität getrennt beobachten.

Quellenbasis: Camunda migration journey; migration strategies and guidance; conceptual differences

KAPITEL 14

Migrationswerkzeuge und Datenmigration

Werkzeuge reduzieren Analyse- und Übertragungsarbeit. Sie ersetzen nicht die Architekturentscheidung und nicht die fachliche Abnahme.

14.1 Werkzeugübersicht

Werkzeug	Zweck	Ergebnis	Nicht automatisch gelöst
Diagram Converter	BPMN- und DMN-Dateien analysieren und konvertieren	Konvertierte Dateien, Aufgabenliste und Auswertung	Fachliche Semantik, Code, UI und Laufzeittests
Code Conversion	Ausgewählte Codekonstrukte bei der Umstellung unterstützen	Ausgangspunkt für refaktorierten Code	Architektur, Transaktion, Idempotenz und Qualitätsprüfung
Data Migrator Runtime	Aktive Instanzen und unterstützte Runtime-Daten übertragen	Neue aktive Instanzen in Camunda 8	Nicht unterstützte BPMN-Zustände und externe Seiteneffekte
Data Migrator History	Historische Daten in unterstützten C8-Sekundärspeicher übertragen	Recherchierbare historische Datensätze	SaaS-Direktmigration, automatische Zusammenführung aller Datensichten

Versionskompatibilität

Für den Referenzstand unterstützt die Migrations-Tooling-Linie 0.3.x Camunda 7.24.x als Quelle und Camunda 8.9.x als Ziel. Die Kompatibilitätsmatrix muss vor jeder Ausführung erneut geprüft werden.

14.2 Diagram Converter systematisch einsetzen

1. Alle BPMN- und DMN-Dateien aus Repository und Deployments sammeln und deduplizieren.
2. Converter zunächst im Analysemodus ausführen und Ergebnisbericht versionieren.
3. Befunde nach Prozess, Elementtyp und notwendiger Aktion aggregieren. Die erzeugte XLSX-Auswertung kann dafür Pivot-Sichten bilden.
4. Automatisch konvertierbare XML-Namespaces und Properties von fachlich zu entscheidenden Punkten trennen.
5. Konvertierte Modelle im Zielmodeler öffnen, linten und gegen die Zielversion deployen.
6. Für jedes Modell einen fachlichen Golden Path und relevante Ausnahmewege testen.
7. Manuelle Änderungen nachvollziehbar dokumentieren, damit spätere Ausgangsversionen reproduzierbar konvertiert werden können.

14.3 Data Migrator Choreografie

Data Migrator: kontrollierte Choreografie



Voraussetzung: vollständige Backups, bekannte Einschränkungen, wiederholbare Testmigration

Migration von Runtime und Historie ist eine Datenoperation, nicht der Ersatz für Modell- und Code-Refactoring.

Abbildung 9: Choreografie einer kontrollierten Datenmigration.

Die Runtime-Migration benötigt Zugriff auf die Camunda-7-Datenbank und auf die Orchestration Cluster APIs. Zielmodelle benötigen einen process-level None Start Event und einen Migrator Execution Listener beziehungsweise validierten Jobtyp. Für die History Migration muss Camunda 7 gestoppt sein und bei aktuellem Stand ein relationaler Camunda-8-Sekundärspeicher direkt erreichbar sein. Dadurch ist dieser History-Pfad Self-Managed vorbehalten.

14.4 Runtime Migration: Vorbereitung

Runtime-Data-Migrator-Checkliste

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Sind alle Zielmodelle vorab deployt und mit gleicher fachlicher Semantik getestet?	Deploymentliste und Testbericht	
2	Besitzt jedes Zielmodell einen process-level None Start Event?	Modellanalyse	
3	Ist der Migrator Listener oder validierte Jobtyp vorhanden?	BPMN-Prüfung	
4	Sind nicht unterstützte aktive BPMN-Zustände identifiziert?	Instanzsegmentierung	
5	Sind Variablentypen, Namen und Transformationen geprüft?	Dry Run und Fehlerbericht	
6	Ist Camunda 7 für den Migrationslauf gestoppt oder schreibgeschützt gemäß Toolvorgabe?	Cutover-Protokoll	
7	Sind vollständige Backups und Restore-Schritte vorhanden?	Restore-Nachweis	
8	Sind Startkanäle und Messages während des Freeze kontrolliert?	Routing- und Queueplan	

14.5 Typische Data-Migrator-Grenzen

Grenze	Folge	Behandlung
History Level unter erforderlichlichem Detail	Notwendige Aktivitätsinformationen fehlen	History Level früh prüfen und Instanzstrategie anpassen
Nicht unterstützte Variablentypen oder Namen	Variable wird abgelehnt oder benötigt Transformation	Transformation konfigurieren oder externisieren
Embedded-Subprocess-Lokalvariablen	Können gemäß aktueller Limitierung ignoriert werden	Vorab inventarisieren und fachlich rekonstruieren
Aktive Multi-Instance-Konstruktionen	Bestimmte Zustände können nicht migrierbar sein	In migrierbaren Zustand überführen, abfließen lassen oder neu starten
Formulare	Nur unterstützte Camunda Forms werden übertragen	Embedded oder eigene Forms separat migrieren
User Operation Log	Nicht alle Betriebs- und Benutzeraktionen werden übertragen	Auditorarchiv oder separaten Nachweis erhalten
Identity-Daten und Passcodes	Keine vollständige Identity-Migration	Identity und technische Credentials neu provisionieren
Runtime und History als getrennte Sichten	Aktive und historische Datensätze können in Operate getrennt erscheinen	Legacy-ID und Verifikationskonzept verwenden
Externe Seiteneffekte	Tool kennt Zahlungen, E-Mails oder Fachtransaktionen nicht	Fachliche Reconciliation

14.6 History Migration und Audit

Entscheidung	Option A: Migrieren	Option B: Archivieren
Suchort	Camunda-8-operative Sicht	Separates revisionssicheres Archiv oder Data Warehouse
Komplexität	Tool, RDBMS-Zugriff, Downtime und Nachprüfung	Export, Viewer und Aufbewahrung
Vollständigkeit	Durch Toolgrenzen und Felder begrenzt	Kann vollständigen Originaldatensatz erhalten
Betrieb	Eine zentrale Oberfläche, aber zusätzliche Datenmenge	Zwei Rechercheorte während Retention
SaaS	Aktueller direkte DB-Pfad nicht verfügbar	Typischer Weg für historische C7-Daten
Recht	Datenminimierung und Löschung in C8 sicherstellen	Archivzugriff, Sperre und Löschung sicherstellen

14.7 Verifikationsplan

Kontrolle	Methode	Akzeptanzbeispiel
Definitionsanzahl	Quelle und Ziel nach Prozess-ID und Version vergleichen	100 Prozent der freigegebenen Modelle
Aktive Instanzen	Zählung nach Prozess und Zustand	Keine unerklärte Abweichung
Variablen	Hash, Stichprobe und Schema-Validierung	Alle Pflichtfelder korrekt, dokumentierte Transformationen
User Tasks	Zählung, Assignment, Due Date, Form und	Alle migrierten Tasks auffindbar und

Kontrolle	Methode	Akzeptanzbeispiel
	Zugriff	bearbeitbar
Timer und Messages	Wartezustände und Testereignisse	Keine verlorenen oder doppelt ausgelösten Fälle
Historie	Stichprobe mit Legacy-ID und Zeitachse	Audit-relevante Datensätze recherchierbar
Fachsystem	Reconciliation externer Buchungen und Status	Keine unkontrollierten Duplikate
Performance	Such- und Migrationsdauer sowie Ressourcen	Innerhalb Cutover-Fenster und Kapazitätsgrenzen

Quellenbasis: *Migration tooling overview; Diagram Converter; version compatibility; Data Migrator runtime, history and limitations*

KAPITEL 15

Was in Camunda 7 nicht mehr neu entstehen sollte

Dieses Kapitel beantwortet die Modernisierungsfrage direkt. Die Priorität richtet sich danach, ob ein Muster in Camunda 8 kein Pendant besitzt, eine andere Semantik hat oder ein unnötiges Betriebsrisiko erzeugt.

15.1 Prioritätsklassen

Klasse	Bedeutung	Handlung
A	Blockiert oder erzwingt Neudesign in Camunda 8	Sofort stoppen und aktiv abbauen
B	Erzeugt hohes Migrations- oder Betriebsrisiko	In laufender Produktpflege modernisieren
C	Konvertierbar, aber mit Test- oder Anpassungsaufwand	Bei nächster Änderung bereinigen
D	Tragfähiges, migrationsfreundliches Muster	Standardisieren und wiederverwenden

15.2 Anti-Pattern-Katalog für Camunda 7

Klasse	Nicht mehr neu bauen	Warum problematisch	Zielmuster
A	Neue CMMN-Lösungen	Camunda 8 hat keine direkte CMMN- Runtime	BPMN, DMN und Fachanwendung neu entwerfen
A	Tiefe ProcessEnginePlugin- oder CommandInterceptor-Anpassungen	Keine gleichwertige Erweiterung im verteilten C8-Kern	Public API, Plattformservice, Worker oder Eventing
A	BpmnParseListener als unsichtbare Modelltransformation	Deploymentverhalten ist nicht portabel und Modell zeigt nicht die Wahrheit	Build-time-Transformation, Element Templates, Modellrichtlinie
A	Direkte SQL-Integration auf ACT_RU-, ACT_HI- oder ACT_RE- Tabellen	Internes Schema und völlig anderes C8-Datenmodell	REST, Client, Export, Event oder eigenes Read Model
A	Embedded Task Forms mit Engine- und Angular-Abhängigkeit	Kein direkter Port und starke Kopplung an C7 Tasklist	Camunda Forms oder eigene Webanwendung
A	Serialisierte Java-Objekte als langfristige Prozessvariablen	Klassenpfad, Sicherheitsrisiko und nicht portabler Vertrag	Versioniertes JSON oder Fachsystemreferenz
A	Geschäftslogik in JUEL-Bean- Aufrufen	FEEL greift nicht auf Spring Beans zu; Logik ist unsichtbar und schwer testbar	Worker, Application Service oder DMN
B	Gemeinsame DB-Transaktion zwischen Delegate und Fachsystem voraussetzen	C8 besitzt keine gemeinsame Transaktionsgrenze	Idempotenz, Outbox und Kompensation
B	Große Dokumente und Listen als Variablen	C8-Payloadlimit und hohe Exportkosten	Object Storage und Referenz
B	Custom Variable Serializer	Kein entsprechender Laufzeitmechanismus im Ziel	Plain JSON-Schema
B	History Event Handler als einzige Unternehmensintegration	C8-Historie und Exportarchitektur unterscheiden sich	Unterstützte Events, Exporter oder Datenpipeline
B	Cockpit Plugin für kritische Geschäftsoperationen	UI-Erweiterung ist nicht portabel	Eigene Operations-App auf Public APIs

Klasse	Nicht mehr neu bauen	Warum problematisch	Zielmuster
B	Spezialvariablen aus DelegateExecution oder Engine-Kontext	Remote Worker erhält nur explizite Daten	Expliziter JSON-Vertrag
B	Unbegrenzte Retry-Schleifen oder immer gleiche Retry-Zeit	Überlastet Downstream und verschleiern permanente Fehler	Begrenzte Retries, Backoff, Incident und Owner
B	Technische Fehler als BPMN Gateway-Pfade modellieren	Vermischt Fachsemantik mit Infrastrukturrauschen	Job Retry und Incident, nur Fachfehler als BPMN
B	Business Calendar ausschließlich als Engine-Plug-in	Nicht portabel und schwer testbar	Expliziter Kalenderdienst oder modellierte Fristberechnung
B	Mehrere externe Seiteneffekte in einem undurchsichtigen Delegate	Nach Wiederholung unklar, welcher Effekt bereits erfolgt ist	Atomare idempotente Schritte oder Saga
B	Secrets oder Tokens in Prozessvariablen	Historisierung und breite Sichtbarkeit	Secret Manager und kurzlebige technische Credentials
C	Komplexe JUEL-Ausdrücke mit Spin	Konvertierung zu FEEL ist fehleranfällig	Einfache FEEL-Ausdrücke und Worker-Mapping
C	Engine-spezifische Implementierungsklassen statt Public API	Minor Updates und Migration brechen leichter	Dokumentierte Interfaces und APIs
C	Neue interne REST-Proxies, die C7-Antworten unverändert extern publizieren	Externe Verbraucher binden sich an auslaufende API	Eigene fachliche API
C	Unklare Call-Activity-Bindings	Nicht reproduzierbare Versionswahl	Versionierungsstandard und Deploymenttest
D	External Task mit sauberem Topic und JSON-Vertrag	Konzept liegt nahe am C8-Worker	Job Worker mit angepasster Semantik
D	Engineunabhängiger Application Service	Geschäftslogik ist portabel und testbar	Hinter Worker wiederverwenden
D	Kleine primitive oder JSON-kompatible Variablen	Gute Interoperabilität	Mit Schema und Budget fortführen
D	Eigene UI hinter fachlichem Backend	Task- und Enginezugriff ist gekapselt	Backend auf C8 APIs umstellen

15.3 Sofortmaßnahmen in einem bestehenden C7-Portfolio

1. Architekturregel beschließen: keine neuen Engine-Internas, keine direkten Tabellenzugriffe und keine serialisierten Java-Objekte.
2. CI-Scans für Delegate Expressions, Spin-Aufrufe, ACT_SQL, ProcessEnginePlugin und Embedded Forms einführen.
3. Variablenmessung aktivieren und Top-Verbraucher nach Größe sowie Typ bereinigen.
4. Geschäftslogik aus JavaDelegates in engineunabhängige Services extrahieren.
5. Neue Integrationen als External Task oder fachlich gekapselte API bauen, sofern C7 noch Zielplattform ist.
6. Neue Formulare auf Camunda Forms oder eigene UI mit Backend ausrichten.
7. CMMN- und Plugin-reiche Lösungen im Portfolio früh priorisieren, da sie keine einfache Endphase erlauben.
8. Laufzeit alter Instanzen erfassen und Prozesse mit mehrjähriger Dauer separat behandeln.

15.4 Was in Camunda 8.9 ebenfalls nicht neu entstehen sollte

Nicht neu verwenden	Grund	Ziel
Operate v1 REST API	Für 8.10 zur Entfernung angekündigt	Orchestration Cluster REST API

Nicht neu verwenden	Grund	Ziel
Tasklist v1 REST API	Für 8.10 zur Entfernung angekündigt	Orchestration Cluster REST API
Job-based User Task für Lifecycle und Query	Task Management ab 8.10 nicht unterstützt	Camunda User Task
Zeebe Java Client	Durch Camunda Java Client ersetzt	Camunda Client
Spring Zeebe SDK	Durch Camunda Spring Boot Starter ersetzt	Camunda Spring Boot Starter
Zeebe Process Test	Deprecated und für Entfernung angekündigt	Camunda Process Test
Numerische Schlüssel in alten API-Objekten	In 8.8 bereits entfernt	String Keys und aktuelle Content Types
Tasklist GraphQL API	In 8.8 entfernt	Aktuelle REST APIs
Interne Komponentenendpunkte oder Datenbankschemata	Keine Public-API-Garantie	Dokumentierte Public APIs
Veraltete einzelne Images bei vereinheitlichter Architektur	Image- und Konfigurationsmodell entwickelt sich weiter	Upgrade-Leitfaden und aktuelle Distribution

15.5 Code-Review-Fragen

Review-Checkliste

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Kann die Geschäftslogik ohne laufende Camunda Engine unit-getestet werden?	Unit-Test-Suite	
2	Ist jeder externe Seiteneffekt idempotent oder kompensierbar?	Design und Test	
3	Verwendet der Prozess nur JSON-kompatible, begrenzte Daten?	Schema und Messung	
4	Sind Enginezugriffe hinter einem Adapter oder Gateway?	Abhängigkeitsgraph	
5	Enthalten Modelle ausschließlich einfache Datenexpressionslogik?	BPMN- und DMN-Lint	
6	Sind Fachfehler und technische Fehler getrennt?	Fehlerkatalog	
7	Verwendet neuer C8-Code keine bereits deprecated Schnittstelle?	Dependency- und API-Scan	

Quellenbasis: *Migration readiness guidance; conceptual differences; Camunda 8 deprecations and public API definition*

KAPITEL 16

Readiness, Cutover, Rollback und Abnahme

Produktionsreife wird nicht durch einen erfolgreichen Deploymenttest belegt. Sie entsteht aus messbaren Nachweisen über Lösung, Daten, Betrieb und Organisation.

16.1 Readiness-Scorecard

Dimension	Gewicht	0 Punkte	1 Punkt	2 Punkte
Modelle	15	Nicht analysiert	Konvertiert, offene Befunde	Fachlich getestet und freigegeben
Code und Integration	20	Enginekopplung unbekannt	Refactoring teilweise	Worker, Idempotenz und Verträge getestet
Daten und Variablen	15	Typen und Größen unbekannt	Inventar vorhanden	Schemas, Budget und Transformation validiert
User Tasks und UI	10	Nicht inventarisiert	Zielkonzept vorhanden	Rollen, Forms und E2E getestet
Betrieb und Security	15	Keine Zielplattform	Plattform aufgebaut	SLO, Backup, Restore und Hardening nachgewiesen
Laufende Instanzen	10	Keine Strategie	Segmentiert	Abfluss oder Migration geprobt
Cutover und Rollback	10	Nur Termin	Runbook vorhanden	Generalprobe mit Abbruchkriterien
Organisation und Support	5	Owner unklar	Rollen benannt	Training, Bereitschaft und Eskalation aktiv

Berechnung: Punkte je Dimension mit Gewicht multiplizieren und durch zwei teilen. Unter 70 Prozent ist ein Produktionscutover nicht freigabefähig. Zwischen 70 und 84 Prozent sind dokumentierte Restmaßnahmen und Risikofreigaben erforderlich. Ab 85 Prozent entscheidet weiterhin das Gate-Gremium anhand kritischer Nullpunkte. Ein Nullpunkt in Backup, Security, Datenkonsistenz oder Instanzstrategie kann nicht durch andere Punkte kompensiert werden.

16.2 Vollständige Readiness-Checkliste

Produktionsfreigabe

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Sind Scope, Zielrelease und Referenzarchitektur eingefroren?	Architecture Decision Record	
2	Sind alle Modelle analysiert, konvertiert und fachlich getestet?	Converter-Bericht und Testreport	
3	Sind Delegates, Listener, Plugins und External Tasks vollständig zugeordnet?	Codeinventar	
4	Sind Variablen JSON-kompatibel, größenbegrenzt und datenschutzbewertet?	Schema- und Variablenkatalog	
5	Sind Worker idempotent und gegen Timeout, Duplikat und Downstream-Ausfall getestet?	Resilienztest	

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
6	Sind Formulare, Tasks, Rollen und eigene UIs E2E getestet?	Abnahmeprotokoll	
7	Sind Start-, Message- und Task-Routing im Parallelbetrieb eindeutig?	Routingtest	
8	Sind Upgrade, Backup und Restore der Zielpattform geprobt?	Betriebsnachweis	
9	Sind Monitoring, Alerts, Logs und Traces aktiv?	Dashboard-Review	
10	Sind Last- und Kapazitätstests mit realistischen Variablen durchgeführt?	Performancebericht	
11	Ist die Strategie für aktive Instanzen pro Prozess beschlossen?	Instanzmatrix	
12	Ist eine Datenmigration mindestens zweimal reproduzierbar geprobt?	Migrationsprotokolle	
13	Sind Zähl-, Hash- und Stichprobenkontrollen automatisiert?	Verifikationsskripte	
14	Sind Freeze, Cutover, Abbruch und Rollback minutengenau geplant?	Runbook	
15	Sind Kommunikations-, Support- und Eskalationswege aktiviert?	Kontakt- und Bereitschaftsplan	
16	Sind Lizenz, Vertrag und Supported Environments bestätigt?	Freigaben	
17	Sind C7-Aufbewahrung, Audit und Abschaltung geplant?	Decommissioning-Plan	

16.3 Cutover-Runbook als Ablauf

Zeitpunkt	Aktivität	Go oder No-Go-Nachweis
T minus 14 Tage	Generalprobe abschließen, Restfehler bewerten	Keine offenen kritischen Defekte
T minus 7 Tage	Change Freeze, Backup- und Rollbackprüfung	Freigegebener Runbookstand
T minus 1 Tag	Plattformkapazität, Credentials, Routing und Teams prüfen	Preflight vollständig
T minus 0	Neue Starts und definierte Schreibwege stoppen	Quiescence bestätigt
T plus 1	Finales Backup und Quellzählung	Backup-ID und Ausgangsreport
T plus 2	Modelle, Konfiguration und Daten migrieren	Schrittprotokoll ohne kritischen Fehler
T plus 3	Automatisierte technische Verifikation	Zählwerte und APIs innerhalb Toleranz

Zeitpunkt	Aktivität	Go oder No-Go-Nachweis
T plus 4	Fachlicher Smoke Test	Start, Task, Message, Worker und Abschluss
T plus 5	Routing auf Camunda 8 freigeben	Go-Entscheidung protokolliert
Stabilitätsfenster	Metriken, Incidents, Fachvolumen und Nutzerfeedback beobachten	Schwellenwerte eingehalten
Nachlauf	C7 nur gemäß Strategie weiterbetreiben oder abschalten	Keine unbeabsichtigten Starts

16.4 Abbruchkriterien

Kategorie	Beispiel für Abbruch oder Rückfall
Daten	Unerklärte Instanz- oder Taskabweichung über festgelegter Toleranz
Fachlichkeit	Golden-Path-Vorgang kann nicht vollständig abgeschlossen werden
Security	Authentisierung, Mandantentrennung oder Berechtigungstest schlägt fehl
Betrieb	Partitionen instabil, Exportlag wächst unkontrolliert oder Restore unklar
Integration	Kritischer Downstream erzeugt Duplikate oder Fehlerrate über Schwelle
Performance	Start-, Job- oder Tasklatenz überschreitet SLO dauerhaft
Support	Kritische Rollen oder Eskalationswege nicht verfügbar

16.5 Rollback-Ebenen

Ebene	Möglichkeit	Grenze
Worker oder Anwendung	Vorheriges Image deployen	Nur wenn Datenvertrag und Prozessversion kompatibel bleiben
Prozessmodell	Neue Starts auf vorherige Version routen	Bereits gestartete Instanzen bleiben auf ihrer Version
Routing	Starts und Messages zurück auf C7 lenken	Nur wenn keine widersprüchlichen C8-Seiteneffekte entstanden sind
Datenmigration	Migration abbrechen und Ziel verwerfen	Benötigt Freeze und unveränderte Quelle
Plattformupgrade	Restore aus Backup	Einfaches Downgrade kann nicht unterstützt sein
Gesamtcutover	C7 wieder freigeben	Reconciliation aller bereits in C8 erzeugten Effekte erforderlich

Rollback ist eine Datenfrage

Das Zurückschalten von DNS oder Routing genügt nicht, sobald Camunda 8 externe Seiteneffekte ausgelöst hat. Vor einem Rückfall muss geklärt sein, welche Zahlungen, Nachrichten, Dokumente oder Fachstatus bereits entstanden sind und wie sie reconciliert werden.

16.6 Abnahme-KPIs

KPI	Vergleich	Beispielziel
Start-Erfolgsrate	C7-Baseline gegen C8-Stabilitätsfenster	Keine signifikante Verschlechterung
End-to-End-Durchlaufzeit	P50, P95 und P99 je Prozess	Innerhalb fachlicher SLO
Worker-Retryrate	Nach Jobtyp und Fehlerklasse	Nur erklärbare temporäre Fehler
Incident-Alter	Offen nach Severity	Kritische Incidents innerhalb Runbookzeit
Task-Such- und Abschlusslatenz	UI und API	Innerhalb Nutzer-SLO
Export- oder Query-Lag	Sekundärsicht gegen Runtime	Unter vereinbarter Schwelle
Dublettenrate	Fachseitige Reconciliation	Null unkontrollierte Seiteneffekt-Dubletten
Datenabweichung	Quelle, Ziel und Fachsystem	Nur dokumentierte Transformationen

16.7 RACI für eine Migrationswelle

Aktivität	Responsible	Accountable	Consulted	Informed
Zielarchitektur	Architekturteam	Produkt- oder Programmleitung	Security, Betrieb, Entwicklung	Fachbereich
Modell und Code	Produktteam	Product Owner	Architektur, QA	Betrieb
Plattform	Plattformteam	Platform Owner	Security, Vendor Support	Produktteams
Datenmigration	Migrationsteam	Programmleitung	DBA, Audit, Produktteam	Support
Fachabnahme	Fachtest	Process Owner	Produktteam, QA	Programm
Cutover	Change Lead	Business Owner	Alle technischen Owner	Nutzer und Support
Rollbackentscheidung	Incident Commander	Business Owner	Architektur, Plattform, Produkt	Stakeholder
C7-Abschaltung	Plattform und Produkt	System Owner	Audit, Legal, Security	Nutzer

Quellenbasis: Camunda migration journey, Data Migrator and upgrade guidance; operational best practices derived from the documented platform boundaries

KAPITEL 17

Übungen, Fallstudie und Wissenstest

Die Übungen übertragen die Konzepte auf eine realistische Bestandslösung. Für Workshops können einzelne Aufgaben getrennt an Architektur, Entwicklung, Betrieb und Fachseite vergeben werden.

17.1 Schulungsformate

Format	Dauer	Bausteine	Ergebnis
Management-Briefing	90 Minuten	Kapitel 1, 2, 12, 13 und 16	Gemeinsames Zielbild und Roadmapentscheidung
Technischer Überblick	Halber Tag	Kapitel 1 bis 4, 6, 8, 12 und 15	Architekturverständnis und erstes Backlog
Migrationsworkshop	Ein Tag	Kapitel 3 bis 16 plus Fallstudie	Bewertete Prozesskohorte und Migrationsstrategie
Engineering-Training	Zwei Tage	Kapitel 5 bis 10, 14, 15 und Übungen	Worker-, Test- und Datenvertragsmuster
Betriebs-Training	Ein Tag	Kapitel 2, 3, 10 bis 16	Upgrade-, Backup-, Monitoring- und Cutover-Runbook
Portfolio-Assessment	Mehrere Workshops	Kapitel 4, 13, 15 und 16	Priorisierte Migrationswellen

17.2 Fallstudie: Order-to-Cash auf Camunda 7

	Ausgangslage Eine Spring-Boot-Anwendung enthält eine eingebettete Camunda-7-Engine. Ein Prozess nimmt Bestellungen an, prüft Bonität, belastet eine Kreditkarte, wartet auf Versand, erzeugt eine Rechnung und fordert bei Ausnahmen manuelle Klärung an. Pro Jahr starten 2,5 Millionen Instanzen. Fünf Prozent laufen länger als 90 Tage.
--	---

Befund	Technische Ausprägung
Zahlung	JavaDelegate schreibt Payment-Tabelle und schließt Engine-Command in derselben Datenbanktransaktion ab
Kundendaten	Variable customer wird als serialisiertes Java-Objekt gespeichert
Bonität	Gateway-Ausdruck ruft <code>creditBean.isEligible(execution)</code> auf
Versand	Message wird über orderId korreliert; Duplikatstrategie ist nicht dokumentiert
Formular	Embedded HTML Task Form mit direktem Zugriff auf Task- und Variablen-APIs
Reporting	BI liest ACT_HI_PROCINST und ACT_HI_VARINST direkt per SQL
Kalender	ProcessEnginePlugin berechnet Arbeitstage
Historie	Sieben Jahre Aufbewahrung, kein separates Archiv
Betrieb	Rolling Updates wurden selten geprobt; letzter Restore-Test liegt 18 Monate zurück

Befund	Technische Ausprägung
Instanzen	Einige aktive parallele Multi-Instance-Subprozesse und offene User Tasks

17.3 Aufgabe A: Risikoanalyse

1. Jeden Befund einer Prioritätsklasse A bis D aus Kapitel 15 zuordnen.
2. Die technische Ursache benennen. Eine reine Aussage wie nicht unterstützt reicht nicht.
3. Auswirkung auf Modell, Code, Daten, UI und Betrieb beschreiben.
4. Eine Zielmaßnahme und einen objektiven Nachweis formulieren.
5. Die drei zuerst zu bearbeitenden Befunde auswählen und die Reihenfolge begründen.

Befund	Klasse	Ursache	Zielmaßnahme	Nachweis

17.4 Aufgabe B: Worker und Transaktion

Entwerfen Sie den Zahlungsschritt neu. Das Payment-System unterstützt einen Idempotency-Key. Die Prozessinstanz kann nach dem erfolgreichen Charge, aber vor dem Complete-Kommando die Verbindung verlieren.

Arbeitsblatt

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Welcher fachliche Schlüssel verhindert eine zweite Belastung?	Definierter Idempotency-Key	
2	In welcher Reihenfolge erfolgen Validierung, Charge, Persistenz und Complete?	Sequenzdiagramm	
3	Welche Fehler sind fachlich, temporär technisch oder permanent technisch?	Fehlerkatalog	
4	Welche Variablen gibt der Worker zurück?	JSON-Ausgabeschema	
5	Wie wird eine Wiederholung nach Netzwerkverlust getestet?	Fault-Injection-Test	
6	Welche Logs und Metriken erlauben Reconciliation?	Observability-Spezifikation	

17.5 Aufgabe C: Instanzstrategie

Entscheiden Sie für drei Kohorten: Instanzen unter 14 Tagen, Instanzen zwischen 14 und 90 Tagen und Instanzen über 90 Tagen. Berücksichtigen Sie offene User Tasks, aktive Multi-Instance-Schritte, Auditpflicht und Parallelbetrieb.

Kohorte	Abfluss	Neustart	Data Migrator	Begründung und Kontrollen
Unter 14 Tage				
14 bis 90 Tage				
Über 90 Tage				

17.6 Aufgabe D: Upgrade-Runbook

Die Self-Managed-Zielplattform läuft auf Camunda 8.8 und soll auf 8.9 aktualisiert werden. Erstellen Sie ein Runbook mit mindestens den folgenden Gates.

Gate	Zu liefernder Nachweis
Eligibility	Ausgangsversion, aktueller Patch, Supported Environments
Releaseanalyse	Breaking Changes, Deprecations, Konfigurationsänderungen
Backup	Vollständigkeit und Restore-Test
Testupgrade	Produktionsnahe Datenmenge und Komponenten
Smoke Test	Deployment, Start, Worker, Message, User Task, Incident, Suche
Abbruch	Messbare Schwellen und Entscheidungsträger
Rollback	Restore-Reihenfolge und Reconciliation
Stabilitätsfenster	Metriken, Dauer und Exit-Kriterium

17.7 Musterlösung zur Fallstudie, gekürzt

Befund	Einordnung	Musterlösung
Gemeinsame Zahlungstransaktion	Klasse B, Semantik ändert sich	Payment mit Idempotency-Key; Application Service; Complete erst nach Fach-Commit; Reconciliation
Serialisiertes customer-Objekt	Klasse A	JSON-Schema mit benötigten Feldern oder customerId als Referenz
creditBean in JUEL	Klasse A	DMN oder eigener Eligibility Worker; FEEL nur für Datenzugriff
Message ohne Duplikatstrategie	Klasse B	Correlation Key, Message ID, TTL und nicht korrelierbare Nachrichten festlegen
Embedded Task Form	Klasse A	Camunda Form oder eigene UI mit Process Gateway und OIDC
SQL auf ACT_HI	Klasse A	Eigenes Read Model über unterstützte APIs oder Datenpipeline; historisches Archiv
Kalender-Plug-in	Klasse B	Expliziter Calendar Service oder deterministische Entscheidung

Befund	Einordnung	Musterlösung
Lange Instanzen	Separate Migrationskohorte	Kurze Instanzen abfließen lassen; lange und komplexe Zustände gegen Toollimits prüfen
Restore-Test veraltet	Betriebliches No-Go	Restore vor Migrationsprobe und Cutover nachweisen

17.8 Wissenstest

Nr.	Frage
1	Ist der Wechsel von Camunda 7 auf Camunda 8 ein In-place-Upgrade?
2	Welche Ursache erzwingt Idempotenz bei Workern?
3	Welche Ausdruckssprache ist in Camunda 8 zentral?
4	Warum sind Spring-Bean-Aufrufe in JUEL migrationsfeindlich?
5	Was geschieht bei Message TTL gleich null ohne offene Subscription?
6	Welche Camunda-7-Datenbankskripte dürfen bei Minor Updates ausgelassen werden?
7	Dürfen Camunda-8-Minor-Releases übersprungen werden?
8	Ist ein Image-Downgrade ein verlässlicher Rollback nach einem C8-Minor-Upgrade?
9	Welches Tool analysiert und konvertiert BPMN und DMN?
10	Welche zwei Hauptbereiche migriert der Data Migrator?
11	Warum ist die aktuelle History Migration Self-Managed vorbehalten?
12	Welche C7-Technik hat kein direktes C8-Pendant?
13	Soll ein PDF als Prozessvariable gespeichert werden?
14	Was ist der Zieltasktyp für neue C8-Human-Workflow-Lösungen?
15	Welche APIs sollten in 8.9 nicht neu verwendet werden?
16	Was ersetzt Zeebe Process Test?
17	Was bedeutet eventual consistency für einen Test?
18	Wann ist Abfluss eine gute Instanzstrategie?
19	Was muss vor einem Rollback reconciliert werden?
20	Welche vier Bereiche bilden das Minimum eines Cutover-Gates?

17.9 Lösungsschlüssel

Nr.	Antwort
1	Nein. Modelle, Code, Architektur und Betrieb werden migriert.
2	Fach-Commit und Complete-Kommando sind getrennt; ein Job kann erneut zugestellt werden.

Nr.	Antwort
3	FEEL.
4	Camunda 8 Expressions arbeiten auf Daten und besitzen keinen direkten Bean-Zugriff.
5	Die Nachricht wird nicht gepuffert und verworfen.
6	Keine erforderlichen Zwischenupdates; die Skripte sind nicht kumulativ.
7	Nein. Upgrades erfolgen sequenziell.
8	Nein. Der dokumentierte Rückfall beruht häufig auf Restore aus Backup.
9	Diagram Converter.
10	Runtime und Historie.
11	Sie benötigt direkten Zugriff auf den relationalen Camunda-8-Sekundärspeicher.
12	Zum Beispiel ProcessEnginePlugin oder BpmnParseListener.
13	In der Regel nein. Extern speichern und Referenz plus Metadaten halten.
14	Camunda User Task.
15	Operate v1 und Tasklist v1 REST APIs.
16	Camunda Process Test.
17	Eine Suchsicht kann einen gerade erzeugten Zustand erst verzögert zeigen; begrenztes Polling ist nötig.
18	Wenn Instanzen kurz und planbar sind und C7 bis zum Ende sicher betrieben werden kann.
19	Bereits ausgelöste externe Seiteneffekte.
20	Lösung, Daten, Betrieb und Organisation.

KAPITEL A

Glossar

Die Begriffe sind auf den Vergleich und die Migration zugeschnitten. Produktdetails sind im konkreten Zielrelease zu verifizieren.

Begriff	Definition
At-least-once	Verarbeitungssemantik, bei der Arbeit erneut zugestellt werden kann, damit sie nicht verloren geht.
Backpressure	Signal oder Mechanismus, der neue Last begrenzt, wenn ein System sie nicht schnell genug verarbeitet.
BPMN Error	Modellierter fachlicher Fehler, der von einem passenden Catch Event behandelt wird.
Business ID	Fachliche Kennzeichnung einer Prozessinstanz in aktuellen Camunda-8-APIs; Migrationsmapping zum C7 Business Key prüfen.
Business Key	Fachliche Kennzeichnung einer Camunda-7-Prozessinstanz.
Camunda Client	Aktueller konsolidierter Client für den Zugriff auf Camunda-8-Orchestration-Cluster.
Camunda Form	Plattformnahes Formularartefakt für User Tasks oder Starts, abhängig vom unterstützten Funktionsumfang.
Camunda Process Test	Aktuelle Testbibliothek für Camunda-8-Prozessdefinitionen und Automationen.
Camunda User Task	Engine-nativer User-Task-Typ für Lifecycle und Query in Camunda 8.
Command	Anforderung, den Runtime-Zustand zu ändern, zum Beispiel Prozessesstart oder Jobabschluss.
Connector	Konfigurierbare Integration, die von einer Connector Runtime ausgeführt wird.
Correlation Key	Wert, der eine Nachricht einer wartenden Prozessinstanz oder Startlogik zuordnet.
Data Migrator	Werkzeug zur Übertragung unterstützter Runtime- und historischer Daten von Camunda 7 zu Camunda 8.
Diagram Converter	Werkzeug zur Analyse und Konvertierung von Camunda-7-BPMN- und DMN-Dateien.
Drain oder Abfluss	Strategie, bei der bestehende C7-Instanzen dort enden und neue Starts nach C8 wechseln.
Eventual Consistency	Eine sekundäre Sicht erreicht den aktuellen Zustand erst nach einer Verzögerung.
Execution Listener	Reaktion auf Lifecycle-Ereignisse eines Prozesselements; Ausführungsmechanismus unterscheidet sich zwischen C7 und C8.
Exporter	Komponente, die Runtime-Ereignisse oder Daten in einen Sekundärspeicher oder ein Zielsystem überträgt.
External Task	Camunda-7-Muster, bei dem externe Worker Aufgaben über Fetch and Lock bearbeiten.
FEEL	Friendly Enough Expression Language, Daten- und Entscheidungsausdrucksprache in Camunda 8 und DMN.

Begriff	Definition
Idempotenz	Eigenschaft, dass eine wiederholte Ausführung keinen zusätzlichen unerwünschten Seiteneffekt erzeugt.
Incident	Operativ sichtbarer Fehlerzustand, der manuelle oder automatisierte Behebung benötigt.
Job	Auszuführende Arbeit, die in Camunda 8 von einem Worker aktiviert und abgeschlossen wird.
Job Type	Vertraglicher Name, über den Worker die passende Arbeit abonnieren.
JUEL	Expression Language, die in Camunda 7 häufig für Daten- und Bean-Zugriffe verwendet wird.
Migration	Plattformwechsel von C7 zu C8 einschließlich Modellen, Code, Betrieb und optional Daten.
Minor Upgrade	Wechsel auf das nächste Minor Release innerhalb derselben Produktlinie.
Orchestration Cluster	Camunda-8-Kern für Prozessausführung, Commands und zustandsbezogene Funktionen.
Outbox	Muster, bei dem Fachänderung und zu versendendes Ereignis lokal atomar gespeichert werden.
Partition	Geordneter Teil des verteilten Runtime-Zustands.
Process Gateway	Eigene Anwendungsschicht, die Camunda-APIs gegenüber Fachsystemen kapselt.
Public API	Dokumentierte Schnittstelle mit definierter Stabilitäts- und Kompatibilitätszusage.
Replication Factor	Anzahl der Replikate eines Partitionszustands zur Ausfallsicherheit.
Retry	Erneuter Ausführungsversuch nach einem technischen Fehler.
Secondary Storage	Speicher für Suche, Betrieb und Historie, der aus Runtime-Daten gespeist wird.
Spin	Camunda-7-Bibliothek und Datenformatintegration, häufig für JSON und XML in JUEL genutzt.
TTL	Time to Live; Zeitraum, in dem eine Message oder ein historischer Datensatz gehalten wird, je Kontext.
Worker	Externer Dienst, der Jobs eines Jobtyps bearbeitet.

KAPITEL B

Komponenten- und Begriffszuordnung

Die Tabelle ist eine Orientierung. Sie behauptet keine vollständige Eins-zu-eins-Entsprechung, weil Architektur und Produktzuschnitt verschieden sind.

Camunda 7	Camunda 8	Wichtiger Unterschied
Process Engine	Orchestration Cluster	Eingebettete Java-Engine gegen remote verteilten Runtime-Kern
Job Executor	Cluster-Jobbereitstellung plus externe Worker	Geschäftscode läuft außerhalb des Runtime-Kerns
External Task Client	Job Worker über Camunda Client	Native Standardsemantik in C8
Cockpit	Operate-Funktionen und Orchestration Cluster APIs	Produkt- und API-Zuschnitt unterscheiden sich
Tasklist	Tasklist-Funktionen und User Task APIs	Camunda User Task als Zieltyp verwenden
Admin	Admin- und Identity-Funktionen	Rollen, Claims und Plattformverwaltung neu konfigurieren
Optimize	Optimize oder externe Analyse	Lizenz, Datenquelle und Release prüfen
C7 REST API	Orchestration Cluster REST API und weitere Public APIs	Endpoint- und Konsistenzmodell ist neu
RuntimeService	Commands über Client oder REST	Keine lokale Java-Service-Schnittstelle
TaskService	User Task APIs	Tasktyp und API-Lifecycle beachten
HistoryService	Search- und Analyse-APIs auf Sekundärspeicher	Eventual Consistency und Retention
JavaDelegate	Job Worker oder Connector	Remote-Ausführung und Wiederholung
Delegate Expression	Job Type und Worker	Bean-Zugriff aus Modell entfernen
ProcessEnginePlugin	Kein direktes Pendant	Anforderung über Public Boundary neu lösen
Embedded Form	Camunda Form oder eigene UI	UI und Authentisierung neu integrieren
CMMN Engine	Kein direktes Pendant	Neu modellieren
Relationale Runtime-DB	Partitionierter Runtime-Kern	RDBMS kann sekundärer Speicher sein, nicht C7-Enginekern

KAPITEL C

Vorlagen für das eigene Projekt

Die folgenden Tabellen können als Ausgangspunkt in Wiki, Spreadsheet oder Architecture Decision Records übernommen werden.

C.1 Prozessinventar

Feld	Beispiel oder Erläuterung
Process ID und Name	Technische ID und fachlicher Name
Owner	Process Owner, Product Owner, verantwortliches Team
Kritikalität	Geschäftsauswirkung und erlaubte Ausfallzeit
Volumen	Starts pro Tag, Peak, aktive Instanzen
Laufzeit	P50, P95, Maximum und Wartezustände
Modellelemente	CMMN, Multi-Instance, Messages, Timer, Call Activities
Code	Delegates, External Tasks, Listener, Plugins, Scripts
Daten	Variablentypen, Größe, Datenschutz, Dokumente
UI	Forms, Tasklist-Nutzung, eigene UIs
Integration	APIs, Messaging, SQL, BI und Downstream-Systeme
Supportstand	C7-Version, JDK, DB, Application Server
Aktive Instanzstrategie	Abfluss, Neustart, Data Migrator, Archiv
Zielwelle	Kohorte, Termin, Abhängigkeiten
Risikoscore	0 bis 4 je Dimension plus Kritikalität

C.2 Architecture Decision Record

Abschnitt	Inhalt
Titel	Kurze, entscheidungsorientierte Bezeichnung
Status und Datum	Vorgeschlagen, akzeptiert, ersetzt oder verworfen
Kontext	Problem, Randbedingungen und betroffene Prozesse
Entscheidung	Gewählte Option mit konkreten Regeln
Ursachen und Treiber	Warum die Option die Anforderungen erfüllt
Alternativen	Mindestens zwei realistische Varianten
Konsequenzen	Positive, negative und organisatorische Folgen
Nachweis	PoC, Test, Benchmark oder Dokumentation
Ablaufdatum	Für Übergangsadapter oder temporäre Parallelstrukturen

C.3 Migrations-Backlog-Eintrag

Feld	Beispiel
Befund	Delegate verwendet gemeinsame Fach- und Engine-Transaktion
Ursache	Embedded Engine teilt Transaction Manager; C8 Worker ist remote
Risiko	Doppelte Zahlung nach Wiederholung
Zielzustand	Idempotenter Payment Service mit Idempotency-Key
Akzeptanzkriterium	Fault-Injection-Test nach Fach-Commit erzeugt nur eine Zahlung
Abhängigkeiten	Payment API, Datenbankconstraint, Monitoring
Owner und Termin	Team und Welle
Evidenz	Link auf Testreport, ADR und Dashboard

C.4 Definition of Done für einen migrierten Prozess

Nr.	Prüffrage	Nachweis oder Ergebnis	Status
1	Fachliche Pfade einschließlich Ausnahmen sind abgenommen.	Testbericht	
2	Modelle besitzen keine ungeklärten Converter-Befunde.	Converter-Report	
3	Worker sind idempotent und observierbar.	Resilienztest und Dashboard	
4	Variablen entsprechen Schema, Budget und Datenschutzregeln.	Schema-Validierung	
5	Forms, User Tasks und Rollen sind E2E getestet.	UI-Abnahme	
6	Alle Integrationen verwenden unterstützte Public APIs.	Architektur- und Code-Scan	
7	Instanzen- und Historiestrategie ist ausgeführt oder terminiert.	Migrationsprotokoll	
8	Runbooks, Support und Ownership sind aktiv.	Betriebsfreigabe	
9	Keine neuen deprecated C8-Schnittstellen werden verwendet.	Dependency- und API-Prüfung	

KAPITEL D

Quellen und weiterführende Dokumentation

Die folgenden offiziellen Quellen bilden die fachliche Basis dieses Handbuchs. Produktdokumentation ist versionsbezogen. Vor Umsetzung ist die Dokumentation des tatsächlichen Zielreleases maßgeblich.

1. [Camunda 7 Enterprise End of Life Extension Announcement](#) EOL, letztes Minor Release und Community-Status
2. [Camunda 8 Release Overview](#) Release- und Maintenance-Daten
3. [Camunda 8.8 Release Announcements](#) Deprecations, API-, Client- und User-Task-Änderungen
4. [Camunda 7 to Camunda 8 Migration Guide](#) Überblick über die Plattformmigration
5. [Conceptual Differences](#) Architektur, Transaktionen, Variablen, Expressions und Erweiterungen
6. [Migration Journey](#) Etappen und Strategien der Migration
7. [Diagram Converter](#) Analyse und Konvertierung von BPMN und DMN
8. [Migration Tooling Version Compatibility](#) Kompatibilität von Quell-, Ziel- und Toolversion
9. [Data Migrator Runtime](#) Voraussetzungen und Ablauf der Runtime-Migration
10. [Data Migrator History](#) Historienmigration und Self-Managed-RDBMS-Voraussetzungen
11. [Data Migrator Limitations](#) Aktuelle Grenzen nach Daten und BPMN-Elementen
12. [Camunda 7 Patch Level Update](#) Patch-Aktualisierung innerhalb eines Minor Releases
13. [Camunda 7 Minor Update](#) Minor Update und sequenzielle Datenbankskripte
14. [Camunda 7 Rolling Update](#) Bedingungen und Ablauf des Rolling Updates
15. [Upgrade to Camunda 8.9](#) Sequenzieller Upgradepfad auf 8.9
16. [Manual Upgrade](#) Manueller Upgradeablauf und Restore-basierter Rückfall
17. [Variables](#) Variablensemantik und Payloadlimit
18. [Messages](#) Message Buffering, TTL und Korrelation
19. [Message Events](#) BPMN-Modellierung und Correlation Key
20. [Testing Process Definitions](#) Testschichten für Prozesslösungen
21. [APIs and Tools](#) Public APIs, Clients und Camunda Process Test
22. [Orchestration Cluster REST Data Fetching](#) Starke und eventual consistency von Endpoints
23. [RDBMS Configuration Overview](#) RDBMS als sekundärer Speicher in 8.9
24. [Camunda Licensing](#) Produktionslizenz für Self-Managed ab 8.6

D.1 Nutzungshinweis

Die Quellenliste ist bewusst auf offizielle Camunda-Dokumentation und Camunda-Ankündigungen beschränkt. Konkrete Implementierungsentscheidungen benötigen zusätzlich die Release Notes, Supported Environments, Security Notices und Lizenzbedingungen des tatsächlich eingesetzten Patchstands.

D.2 Dokumentabschluss

Abschlussformel für die Praxis

Zuerst Systemgrenzen und Semantik klären, dann Modelle und Code konvertieren. Zuerst Backups und Nachweise proben, dann Daten und Verkehr umschalten. Eine technisch erfolgreiche Migration ist erst abgeschlossen, wenn Fachprozess, Betrieb, Audit und Abschaltung gemeinsam funktionieren.

Ende des Schulungshandbuchs

Referenzstand 23. Juni 2026